

АННОТАЦИЯ

Выпускная квалификационная работа посвящена разработке веб-приложения для изучения языков с иероглифической письменностью — японского и китайского. Работа выполнена по теме: «Разработка веб-приложения для изучения языков с иероглифической письменностью».

Целью работы является создание веб-платформы, обеспечивающей автоматическую обработку пользовательских запросов в рамках выбранного языкового профиля (японского или китайского) и предоставляющей унифицированный интерфейс морфологического анализа, словарного поиска и AI-объяснений грамматических конструкций.

В работе применяются следующие методы: обработка естественного языка (NLP), морфологический анализ с использованием библиотек SudachiPy [26] (японский) и HanLP [41; 42] (китайский), клиент-серверное проектирование на основе архитектурного паттерна BFF, реляционное моделирование данных, функциональное и нагрузочное тестирование программного обеспечения.

Практическим результатом работы являются: функционирующее веб-приложение, реализованное как прогрессивное веб-приложение (PWA) на основе Next.js 14 (TypeScript [44]) с адаптивным интерфейсом, и Python-сервис FastAPI, выполняющий морфологический анализ и интеграцию с внешними API (Jisho, CC-CEDICT, OpenRouter).

Элементами научно-технической новизны являются: интеллектуальный эвристический классификатор типа запроса, автоматически разграничивающий четыре класса входных данных (kanji, hanzi, sentence, reverse) без участия пользователя; объединённый поиск по японскому, китайскому и английскому языкам в рамках единого интерфейса при нативной обработке английского ввода в обоих языковых профилях..

Работа имеет практическую значимость для самостоятельного изучения японского и китайского языков, а также для использования в системе дополнительного образования и на языковых курсах

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ	8
1.1 Анализ предметной области.....	8
1.2 Постановка задачи и требования к системе	17
1.3 Обзор и выбор технологий	24
1.4 Обзор и сравнение аналогов.....	32
1.5 Выводы по главе 1	37
2 ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ.....	38
2.1 Архитектура системы и модель данных	38
2.2 Проектирование NLP-модуля.....	44
2.3 Проектирование пользовательского интерфейса	53
2.4 Реализация серверной части	58
2.5 Реализация клиентской части	64
2.6 Развёртывание и эксплуатация	70
2.7 Выводы по главе 2	75
3 ЭКСПЛУАТАЦИОННАЯ ЧАСТЬ.....	77
3.1 Анализ результатов и оценка внедрения.....	77
3.2 Тестирование	81
3.3 Лицензирование и защита интеллектуальной собственности.....	87
3.4 Аспекты информационной безопасности	91
3.5 Выводы по главе 3	95
ЗАКЛЮЧЕНИЕ	97
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	99

ВВЕДЕНИЕ

Актуальность

В последние годы наблюдается устойчивый рост интереса к изучению языков с иероглифической письменностью — японского и китайского — среди русскоязычной аудитории. По данным Японского фонда (The Japan Foundation), по состоянию на 2022 год число изучающих японский язык в мире превысило 3,8 миллиона человек, а количество сдающих экзамен JLPT (Japanese-Language Proficiency Test) ежегодно прирастает на 5–7 % [1]. Аналогичную динамику демонстрирует и рынок изучения китайского языка: по данным Hanban (Институт Конфуция), число участников экзамена HSK (Hanyu Shuiping Kaoshi) в 2023 году достигло более 6 миллионов человек по всему миру, что свидетельствует о массовом и нарастающем спросе [2]. Параллельно глобальный рынок образовательных технологий (EdTech) демонстрирует исключительно высокие темпы роста: по оценке аналитического агентства HolonIQ, его объём к 2025 году превысит 404 миллиарда долларов США, причём сегмент изучения иностранных языков составляет одну из наиболее динамично развивающихся его ниш [3].

Вместе с тем изучение японского и китайского языков сопряжено с фундаментальными трудностями, принципиально отличающими данные языки от европейских. Иероглифические системы письма — кандзи в японском и ханьцзы в китайском — насчитывают тысячи графических единиц, каждая из которых обладает самостоятельным значением, чтением и набором вариантов написания. Для достижения уровня базовой грамотности требуется усвоение не менее 2000–3000 иероглифов, а понимание письменного текста невозможно без одновременного владения грамматическими конструкциями, лексикой и фонетической системой — иными словами, все аспекты языка находятся в тесной взаимозависимости. Существующие цифровые инструменты для изучения данных языков — такие как Duolingo, Anki, WaniKani и Jisho — решают отдельные педагогические задачи, однако не

обеспечивают интегрированного подхода: пользователь вынужден одновременно использовать несколько несвязанных между собой приложений для работы со словарём, морфологическим разбором, карточками и системой интервальных повторений по принципу распределённой практики [14]. При этом русскоязычный сегмент рынка остаётся практически неохваченным: подавляющее большинство специализированных инструментов ориентировано на англоязычную аудиторию, а русскоязычные аналоги либо ограничены по функциональности, либо не предоставляют возможностей автоматической обработки текста.

Таким образом, существует реальная практическая потребность в разработке веб-приложения, которое объединяло бы функции интеллектуального словаря, морфологического анализатора и учебной среды в единую русскоязычную платформу. Разработка подобной системы представляется актуальной с технической, педагогической и рыночной точек зрения.

Цель работы

Целью настоящей выпускной квалификационной работы является разработка веб-платформы изучения иероглифических языков с поддержкой языковых профилей (японского и китайского) и нативной обработкой английских запросов, обеспечивающей автоматический морфологический анализ и контекстный словарный поиск без необходимости явного выбора режима поиска.

Задачи работы

Для достижения поставленной цели в рамках работы решаются следующие задачи:

- 1) провести анализ предметной области изучения иероглифических языков, включая исследование лингвистических особенностей японского и китайского письма, характеристик целевой аудитории и существующих педагогических подходов;

- 2) выполнить сравнительный анализ существующих программных аналогов с целью выявления их функциональных ограничений и обоснования необходимости разработки нового решения;
- 3) сформулировать функциональные и нефункциональные требования к разрабатываемому веб-приложению;
- 4) спроектировать клиент-серверную архитектуру системы, включая структуру базы данных, программный интерфейс (API) и схему взаимодействия компонентов;
- 5) реализовать серверную часть приложения с использованием современного технологического стека;
- 6) разработать клиентскую часть приложения с пользовательским интерфейсом, отвечающим требованиям удобства и доступности;
- 7) реализовать модуль автоматической обработки естественного языка, обеспечивающий морфологический анализ текстов на японском и китайском языках и классификацию типа пользовательского запроса;
- 8) провести функциональное тестирование разработанной системы в соответствии с программой и методикой испытаний.

Объект и предмет исследования

Объектом исследования является процесс изучения иероглифической письменности средствами веб-технологий, рассматриваемый с точки зрения организации учебной деятельности пользователя, структуры лексикографических данных и педагогической эффективности цифровых инструментов.

Предметом исследования служат методы автоматической обработки и анализа текста на японском и китайском языках в составе веб-приложения: алгоритмы токенизации [50] и морфологического разбора, методы классификации типа запроса, а также подходы к интеграции результатов лингвистического анализа в пользовательский интерфейс.

Методы исследования

В ходе выполнения работы применяются следующие методы. Для анализа и обработки текстов на иероглифических языках используются методы обработки естественного языка (Natural Language Processing, NLP), в частности морфологический анализ с применением специализированных библиотек токенизации. Для проектирования системы применяется клиент-серверный архитектурный подход, предполагающий чёткое разделение серверной логики и клиентского представления. Структура хранимых данных определяется посредством реляционного моделирования данных. Качество разработанного программного обеспечения оценивается с применением методов функционального тестирования.

Элементы новизны

Научная и техническая новизна работы определяется двумя ключевыми аспектами. Первым из них является разработка интеллектуального классификатора типа запроса, способного автоматически определять тип введённого пользователем текста — является ли он отдельным иероглифом (kanji/hanzi), словом (word), предложением (sentence) или обратным запросом на английском языке (reverse) — без какого-либо участия пользователя в выборе режима поиска. Вторым элементом новизны является поддержка нативного английского ввода в обоих языковых профилях: пользователь выбирает язык однократно при первом запуске, а английские запросы обрабатываются автоматически в рамках активного профиля без дополнительной настройки.

Практическая значимость

Разработанное веб-приложение обладает выраженной практической значимостью как для самостоятельного изучения японского и китайского языков, так и для использования в образовательном процессе под руководством преподавателя. Единая интегрированная среда, объединяющая морфологический анализ, словарный поиск и учебные функции на русскоязычном интерфейсе, позволяет существенно снизить когнитивную нагрузку, связанную с необходимостью параллельного использования

множества разрозненных инструментов. Результаты работы могут быть применены в системе дополнительного образования, на языковых курсах и в форматах дистанционного обучения.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1 Анализ предметной области

Особенности иероглифических систем письма

Иероглифическая письменность представляет собой один из наиболее сложных типов письма, исторически сформировавшихся в Восточной Азии. В отличие от алфавитных систем, в которых графические знаки соответствуют фонемам, иероглифы являются логограммами: каждый знак несёт самостоятельное смысловое наполнение и может обладать несколькими вариантами произношения в зависимости от контекста [4]. В настоящей работе рассматриваются две иероглифические системы — японская и китайская, — которые при значительном историческом сходстве обнаруживают принципиальные различия в структуре, функциях и педагогических требованиях.

Японская письменность является уникальной гибридной системой, объединяющей три взаимодополняющих сценария одновременно. Первый из них — кандзи (漢字) — представляет собой заимствованные из китайского языка иероглифы, адаптированные к японской фонетике и семантике. Каждый кандзи, как правило, обладает двумя классами чтения: онным (音読み, on'yomi), восходящим к китайскому произношению эпохи заимствов

ания, и кунным (訓読み, kun'yomi), отражающим
исконно японское слово с тем же значе-
нием. Количество вариантов чтения о-
дного иероглифа варьируется от одно-
го до десяти и более: так, знак 生 имее-
т онные чтения sei и shō, а также кунные
чтения nama, ki, и ряд других, реализующ-
ихся в составе различных слов. Второ-
й сценарий – хирагана (ひらがな) – являетс-
я слоговой азбукой, используемой для
записи грамматических аффиксов, слу-
жебных слов и слов, не имеющих устояв-
шейся кандзи-записи. Третий – катакан-
а (カタカナ) – функционально идентичная с-
логовая азбука, применяемая преимущ-
ественно для записи заимствованных
слов иностранного происхождения и о-
номатопеэтики. Следствием такой стр-
уктуры является то, что для чтения об-
ычного японского текста учащийся до-
лжен одновременно владеть тремя сце-
нариями, каждый из которых требует о-
тдельного освоения [5; 50].

□□□□□□□□ □□□□□□□□□□ □□□□□□ □□ □□□□□□ □□□□□
□□ □□□□□□□□□□ — □□□□□□□ (汉字 / 漢字). □ □□□□□□□ □□ □□□□□□

中国, 中国是一个国家 中国 是 中国的一个国家 中国是一个国家 中国
中国 中国 是 中国的一个国家 中国; 中国 中国 中国 中国
中国 中国 中国 中国 中国 中国 中国 中国 (拼音)
— 中国 中国 中国 中国 中国, 中国 中国 中国 中国
中国 中国 中国 1958 中国 中国 中国 中国 中国 中国 中国
中国 中国 [6]. 中国 中国 中国 中国 中国 中国 中国
中国 中国 中国 中国 中国: 中国 中国 中国 中国
中国 中国 中国 中国 中国 (中国, 中国, 中国
中国-中国, 中国) 中国 中国 中国, 中国
中国 中国 中国 中国 中国. 中国, 中国 ma 中国
中国 中国 中国 中国 «中国» (māo), «中国»
(má), «中国» (mā) 中国 «中国» (mà). 中国 中国 中国 中国
中国 中国 中国 中国 中国 中国 中国 中国
中国 中国: 中国 中国 中国 (简化字, jiǎnhuàzì), 中国
中国 中国 中国 中国 中国 中国, 中国 中国 中国 中国
中国 中国 (繁体字, fántǐzì), 中国 中国 中国 中国 中国
中国. 中国, 中国 中国 中国 中国 中国 中国 中国
中国 中国 中国 中国 中国, 中国 中国 中国 中国 中国
中国 中国 [7].

Сравнительная характеристика двух рассматриваемых систем письма приведена в таблице 1.1.

Таблица 1.1 – Сравнительная характеристика японской и китайской письменных систем

Признак	Японский язык	Китайский язык
Системы письма	Кандзи, хирагана, катакана (три системы одновременно)	Ханьцзы (иероглифическое письмо); параллельно — пиньинь как фонетическая транслитерация
Число иероглифов для базового уровня	~2 000 (уровень JLPT N1)	~2 500 (уровень HSK 4–5)
Число вариантов чтения одного знака	2–10+ (онные и кунные чтения)	1–3 (варьируются по диалектам и регистрам)
Тональность	Отсутствует (морфологические роли передаются агглютинативной грамматикой)	4 тона + нейтральный (обязательны для правильного понимания)
Упрощённые / традиционные формы	Частичное упрощение исторически принято; обе формы встречаются в текстах	Две параллельные нормы: упрощённые (КНР, Сингапур) и традиционные (Тайвань, Гонконг)
Порядок слов	SOV (субъект — объект — глагол)	SVO (субъект — глагол — объект), ближе к европейским языкам
Пространство между словами	Отсутствует; границы слов определяются морфологически и контекстуально	Отсутствует; сегментация требует NLP-инструментов

Признак	Японский язык	Китайский язык
Экзаменационная система	JLPT (уровни N5–N1)	HSK (уровни 1–6) / HSK 3.0 (уровни 1–9)

Данные таблицы 1.1 наглядно демонстрируют, что при общем иероглифическом происхождении две системы обнаруживают существенные различия в фонетической организации, морфологии и стандартах письма, что обуславливает необходимость отдельной, но взаимосвязанной программной поддержки при разработке единого лингвистического инструмента.

Сложности освоения иероглифических языков

Освоение иероглифических языков сопряжено с рядом трудностей, которые могут быть структурированы по нескольким измерениям: объём обязательного словаря, грамматическая морфология и контекстная многозначность знаков. Понимание данных трудностей необходимо для обоснования функциональных требований к разрабатываемой системе.

Объём обязательного словаря является одним из наиболее значимых барьеров при изучении иероглифических языков. Японский экзамен JLPT (Japanese-Language Proficiency Test) на высшем уровне N1 предполагает знание около 2 000 кандзи и не менее 10 000 лексических единиц; уровень N5 (начальный) ограничен примерно 100 кандзи и 800 словами [8]. Китайский стандарт HSK (Hanyu Shuiping Kaoshi) в шестиуровневой версии на высшем уровне HSK 6 требует владения порядка 5 000 слов и 2 500 иероглифов; новая девятиуровневая версия HSK 3.0, введённая с 2021 года, расширяет требования высшего уровня до 11 000 слов [9]. При этом для уверенного чтения газетного текста [49] на обоих языках требуется знание не менее 2 000–3 000 иероглифов — объём, который при традиционных методах изучения (бумажные карточки, заучивание списков) достигается в среднем за три — пять лет интенсивных занятий.

Контекстная многозначность иероглифов представляет самостоятельную когнитивную проблему. Один и тот же знак может выступать в разных значениях и с разным произношением в зависимости от лексического окружения, грамматической функции и регистра текста. В японском языке, например, иероглиф 行 читается как *gyō* в слове 行列 (*gyōretsu*, «очередь»), как *kō* в 銀行 (*ginkō*, «банк») и как *i-* в глаголе 行く (*iku*, «идти»). В китайском языке аналогичный знак 行 имеет чтения *xíng* («идти», «выполнять»), *háng* («ряд», «торговая организация») и *héng* в устаревших и диалектных контекстах [7]. Такая многозначность делает изолированное заучивание знаков заведомо недостаточным: понимание требует анализа контекста, что, в свою очередь, предполагает наличие инструментов морфологического и семантического разбора.

Грамматическая морфология также существенно усложняет процесс освоения. Японский язык является агглютинативным: глагольные окончания, падежные частицы и модальные суффиксы меняют смысл предложения кардинально, при этом в письменном тексте они записываются хираганой вперемешку с кандзи. Китайский язык, напротив, является изолирующим — слова практически не изменяются по форме, однако грамматические

отношения передаются строгим порядком слов и служебными словами, нередко омографичными с однозначной лексикой. В обоих случаях автоматическое определение границ слов (токенизация) нетривиально, поскольку пробелы в записи отсутствуют, а корректная сегментация невозможна без применения статистических или нейросетевых методов NLP, в том числе рекуррентных архитектур [40] [10].

Сравнительная характеристика основных типов сложностей при изучении двух языков приведена в таблице 1.2.

Таблица 1.2 – Сравнительная характеристика сложностей изучения японского и китайского языков

Сложность	Японский	Китайский
Объём обязательного словаря	~2 000 кандзи (JLPT N1), ~10 000 лексических единиц	~5 000 слов (HSK 6), ~2 500 иероглифов
Многозначность знаков	高 (высокий, дорогой, почтённый) — значение зависит от контекста и чтения	好 (hǎo — хороший, hào — любить) — омографы с разными тонами
Морфологические сложности	Агглютинативные окончания, частицы, глагольные формы	Тональные пары, классификаторы, счётные слова
Необходимость контекста	Критична: 生 читается sei/shō/nama/ki/u- в зависимости от слова	Критична: 行 читается xíng/háng/héng в разных значениях

Модель процесса изучения языка без средств автоматизации (AS-IS)

Для обоснования необходимости разрабатываемой системы необходимо описать текущее состояние учебного процесса учащегося, осваивающего иероглифический язык без специализированных интегрированных инструментов. Данная модель обозначается в практике системного анализа как AS-IS («как есть») и служит отправной точкой для выявления проблем и формирования требований к целевой системе.

В типичном сценарии учащийся, встретив в тексте незнакомое слово или иероглиф, вынужден осуществить поиск вручную, последовательно обращаясь к нескольким разнородным источникам. Для японского языка это может быть онлайн-словарь Jisho, корпус BCCWJ, приложение Takoboto или бумажный словарь; для китайского — Plesco, MDBG, CC-CEDICT или ресурсы Института Конфуция. Поскольку перечисленные источники не интегрированы между собой и имеют различные интерфейсы, пользователь тратит значительное время на переключение между приложениями и вручную сопоставляет полученные данные [11].

После нахождения значения учащийся, как правило, вручную вносит новую единицу в личный словарь или систему карточек (например, Anki). Однако данный шаг нередко пропускается ввиду трудоёмкости, что приводит к повторному поиску того же слова при следующей встрече в тексте. Даже при систематическом занесении слов в карточки учащийся оказывается лишён контекстной информации: изолированная карточка «кандзи — значение» не обеспечивает понимания синтаксических паттернов, в которых данный знак встречается, и не демонстрирует его поведения в составе производных слов.

Отдельную проблему представляет отсутствие систематических повторений. Исследования в области когнитивной психологии убедительно демонстрируют, что долгосрочное запоминание требует интервальных повторений по кривой забывания Эббингауза; однако учащийся без автоматизированной поддержки либо повторяет материал хаотично, либо вовсе отказывается от регулярных повторений ввиду отсутствия напоминаний

[12]. Следствием является высокий процент «возврата» — слов, которые учащийся вынужден искать повторно после периода неиспользования.

Процесс изучения нового слова в модели AS-IS формализован в виде диаграммы IDEF0, представленной на рисунке 1.1. Диаграмма отражает текущий формализованный процесс изучения слова или иероглифического символа без автоматизации.

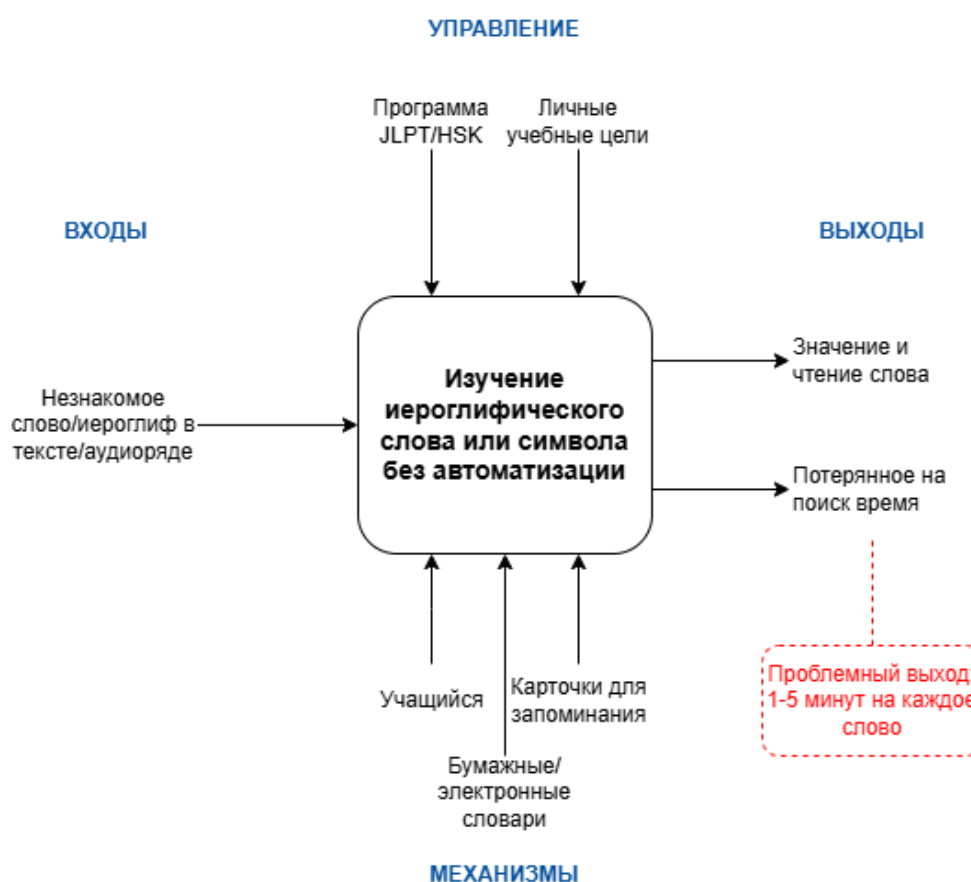


Рисунок 1.1 – Контекстная диаграмма IDEF0 уровня A0 процесса изучения иероглифического языка без автоматизации (AS-IS)

Декомпозиция контекстной диаграммы до уровня A1–A5 (рисунок 1.4) детализирует подпроцессы. Блок A1 фиксирует встречу незнакомого слова; A2 и A3 отражают итеративный поиск по нескольким словарям (при промахе в одном источнике учащийся переходит к следующему); A4 представляет ручную фиксацию значения в личные записи или карточки Anki; A5 — попытку запоминания без системы интервальных повторений. На диаграмме

выделены три уязвимости: отсутствие единого формата записи [38], ручная агрегация нескольких трат и зависимость от готовности учащегося к самодисциплине.

Анализ диаграммы позволяет выделить три ключевые проблемные точки модели AS-IS. Первая — избыточные временные затраты на поиск: переключение между несколькими приложениями занимает от одной до нескольких минут на каждое слово, что при частоте новых единиц в аутентичном тексте (от 10 до 30 незнакомых слов на страницу у учащихся среднего уровня) делает процесс чтения крайне медленным.

Вывод

Проведённый анализ предметной области позволяет сформулировать следующий вывод. Иероглифические системы письма японского и китайского языков характеризуются высокой сложностью, обусловленной объёмом обязательного словаря, многозначностью знаков и отсутствием пробелов между словами. Существующие инструменты для изучения данных языков решают эти проблемы лишь частично: они либо ограничены одним языком, либо не предоставляют интегрированного сочетания словарного поиска, морфологического анализа и системы повторений, либо не имеют русскоязычного интерфейса. Модель AS-IS наглядно демонстрирует, что без средств автоматизации процесс изучения сопряжён с высокими временными затратами. Совокупность указанных факторов обосновывает необходимость разработки единой веб-платформы, интегрирующей интеллектуальный поиск, NLP-анализ текста и учебные функции в рамках единого русскоязычного интерфейса.

1.2 Постановка задачи и требования к системе

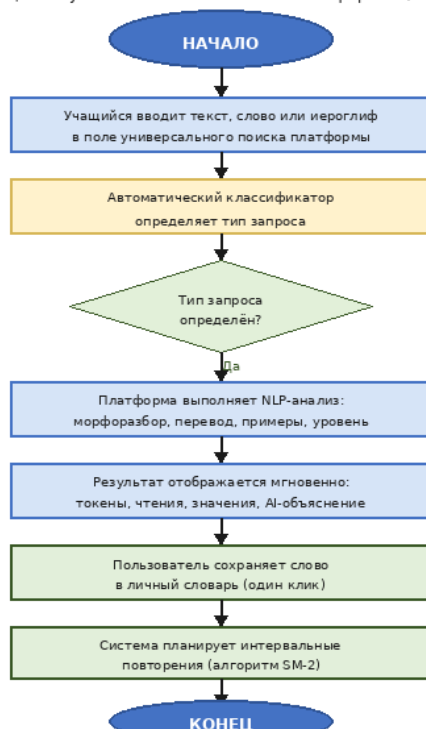
Целевая модель процесса изучения языка

На основании проблем, выявленных в параграфе 1.1, сформулировано целевое состояние учебного процесса. В отличие от существующей практики,

когда учащийся последовательно обращается к нескольким разрозненным инструментам, целевая модель предполагает единую точку входа: пользователь выбирает языковой профиль (японский или китайский) при первом запуске, после чего вводит произвольный текст, слово или иероглиф в поле поиска. Платформа автоматически определяет тип запроса и возвращает структурированный результат в пределах двух секунд. Мгновенный морфологический разбор с переводами, информация об иероглифах и оценка уровня сложности по JLPT/HSK предоставляются без переключения между приложениями. Одним нажатием слово добавляется в личный словарь, после чего система включает его в план интервальных повторений [13].

Диаграмма деятельности целевого процесса представлена на рисунке 1.2.

Процесс изучения нового слова с веб-платформой (TO-BE)



Функциональные требования

На основании анализа предметной области и сравнения с существующими аналогами сформулированы функциональные требования к разрабатываемой системе, представленные в таблице 1.3.

Таблица 1.3 – Функциональные требования к веб-платформе

№	Иден-тифи-катор	Описание требования	Приоритет
1	ФТ-1	Система принимает ввод на языке активного профиля (японском или китайском), а также на английском языке и в форматах ромадзи/пиньинь; английский язык распознаётся нативно без дополнительной настройки.	Высокий
2	ФТ-2	Система автоматически классифицирует тип запроса в рамках выбранного языкового профиля: иероглиф (kanji/hanzi), слово (word), предложение (sentence) или обратный запрос на английском (reverse); английский язык поддерживается нативно в обоих профилях.	Высокий
3	ФТ-3	Система выполняет морфологический анализ японского текста: сегментирует на токены, определяет части речи, исходные формы (леммы), чтения в хирагане и переводы каждого токена.	Высокий
4	ФТ-4	Система выполняет морфологический анализ китайского текста: сегментирует на слова, определяет части речи, пиньинь с тонами и переводы лексических единиц.	Высокий
5	ФТ-5	Система обеспечивает поиск по словарной базе в трёх режимах: точный поиск по иероглифу или слову, нечёткий поиск по похожим написаниям на основе расстояния редактирования [54], семантический поиск по смыслу запроса.	Высокий
6	ФТ-6	Система предоставляет инспектор иероглифа: значения, онные и кунные чтения (японский) /	Высокий

№	Идентификатор	Описание требования	Приоритет
		пиньинь (китайский), радикалы, порядок черт, уровень JLPT/HSK и примеры употребления.	
7	ФТ-7	Система определяет преобладающий уровень сложности текста по шкале JLPT (N5–N1) или HSK (1–6) и отображает статистику распределения слов по уровням.	Средний
8	ФТ-8	Система формирует объяснения грамматических конструкций, выявленных в тексте, с использованием языковой модели: наименование, правило, примеры, пояснение на русском языке.	Средний
9	ФТ-9	Зарегистрированный пользователь сохраняет слова и иероглифы в личный словарь, отмечает прогресс освоения и просматривает историю поиска.	Средний
10	ФТ-10	Система функционирует как прогрессивное веб-приложение (PWA): адаптивный интерфейс для мобильных и настольных устройств, установка на домашний экран, базовая работа офлайн.	Низкий

Центральным с точки зрения новизны является ФТ-2: автоматическая классификация типа запроса. Классификатор разграничивает четыре класса входных данных — отдельный иероглиф (kanji/hanzi), слово (word), связный текст (sentence) и обратный запрос на английском языке (reverse) — без каких-либо действий со стороны пользователя. Язык запроса не определяется классификатором: он задаётся языковым профилем, который пользователь выбирает при первом запуске. ФТ-2 является обязательным предшествующим шагом для ФТ-3 и ФТ-4, что отражено на диаграмме вариантов использования.

Нефункциональные требования

Нефункциональные требования к системе приведены в таблице 1.4.

Таблица 1.4 – Нefункциональные требования к веб-платформе

№	Категория	Требование
1	Производительность	Время отклика на запрос морфологического анализа не превышает 2 с при объёме текста до 500 символов. Время первоначальной загрузки интерфейса — не более 3 с при скорости соединения 10 Мбит/с.
2	Масштабируемость	Архитектура допускает горизонтальное масштабирование серверной части без изменения кода. Хранилище поддерживает не менее 100 000 словарных записей без деградации производительности поиска.
3	Безопасность	Аутентификация через JWT: время жизни токена доступа — не более 15 минут, обновление через refresh-токен. Все коммуникации — исключительно по HTTPS. Пароли хранятся в хэшированном виде (bcrypt, cost factor ≥ 12).
4	Доступность и надёжность	Целевой показатель доступности (SLA) — не менее 99%. Серверная часть корректно обрабатывает ошибки сторонних сервисов и возвращает частичный результат вместо полного отказа.
5	Кроссбраузерность	Корректное отображение и работа в Chrome ≥ 110 , Firefox ≥ 110 , Safari ≥ 16 , Edge ≥ 110 на Windows 10+, macOS 12+, iOS 15+ и Android 10+. Корректный рендеринг CJK-символов на всех платформах.
6	Сопровождаемость	Разделение ответственности между модулями, документация в формате docstring/JSDoc, покрытие модульными тестами не менее 60% для серверной части. Деплой без простоя через CI/CD-конвейер.

Отдельного внимания заслуживает требование кроссбраузерности в части отображения иероглифических символов CJK. Корректный рендеринг

зависит от наличия соответствующих шрифтов в операционной системе, поэтому в составе клиентской части подключается веб-шрифт с полным СЖК-покрытием через механизм CSS @font-face, обеспечивающий идентичное отображение знаков вне зависимости от локальных настроек устройства.

Диаграмма вариантов использования

Функциональная модель системы с точки зрения взаимодействия участников формализована в виде диаграммы вариантов использования (рисунок 1.7). В системе выделены два пользовательских актора — незарегистрированный и зарегистрированный пользователь — и два внешних: NLP-сервис и языковая модель. Незарегистрированный пользователь имеет доступ к функциям поиска и анализа (ФТ-1 — ФТ-7). Зарегистрированный пользователь, помимо перечисленного, работает с личным словарём (ФТ-9) и AI-объяснениями (ФТ-8). ФТ-2 включает (<<include>>) ФТ-3 и ФТ-4, поскольку классификация типа запроса предшествует морфологическому анализу.

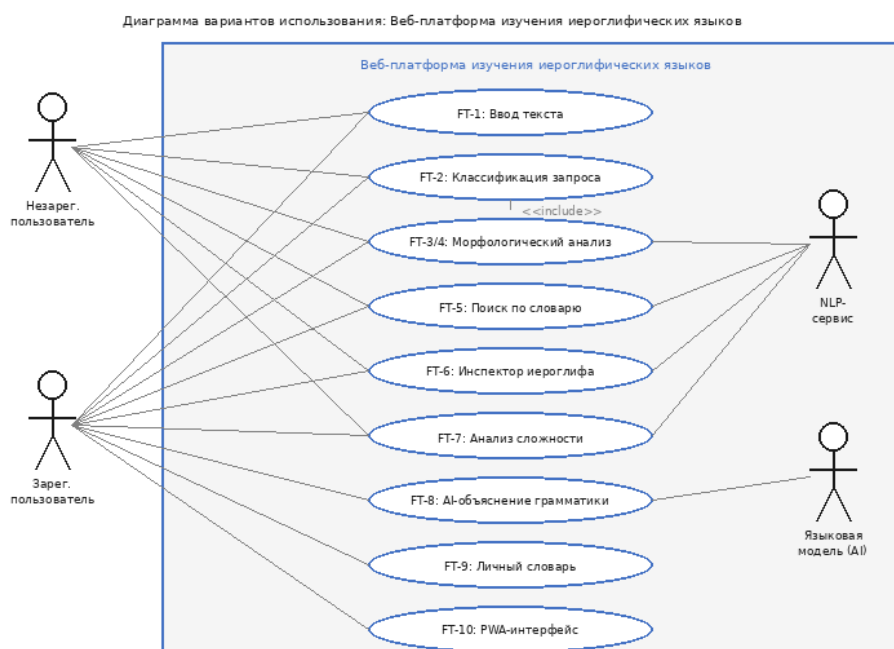


Рисунок 1.7 – Диаграмма вариантов использования веб-платформы изучения иероглифических языков

Выводы по параграфу

Сформулированы 10 функциональных и 6 нефункциональных требований. Целевая модель процесса изучения и диаграмма вариантов использования определяют границы разрабатываемой системы и служат исходной спецификацией для проектирования архитектуры, рассматриваемого в главе 2.

Метод приоритизации требований MoSCoW

Для расстановки приоритетов функциональных требований применяется метод MoSCoW, широко используемый в гибких методологиях разработки программного обеспечения [15]. Аббревиатура расшифровывается как Must Have (обязательно должно быть), Should Have (желательно), Could Have (возможно при наличии ресурсов) и Won't Have this time (не включается в данную версию). Применительно к разрабатываемой системе: требования ФТ-1 — ФТ-6 относятся к категории Must Have и составляют минимально жизнеспособный продукт (MVP, Minimum Viable Product); ФТ-7 и ФТ-8 — к категории Should Have; ФТ-9 и ФТ-10 — к категории Could Have.

Взаимосвязи между требованиями

Между функциональными требованиями существуют зависимости, влияющие на порядок реализации. ФТ-2 (классификатор) является предусловием для ФТ-3, ФТ-4 и ФТ-5: без определения типа запроса невозможно выбрать правильный обработчик. ФТ-6 (инспектор иероглифа) зависит от ФТ-3 и ФТ-4: карточка иероглифа формируется в рамках морфологического анализа. ФТ-7 (анализ сложности) зависит от ФТ-3/ФТ-4: уровень сложности вычисляется по уровням токенов, полученных при разборе. ФТ-8 (AI-объяснения) зависит от ФТ-3/ФТ-4: для формирования грамматического объяснения языковой модели передаётся разобранная структура предложения. ФТ-9 (личный словарь) зависит от авторизации, предусмотренной НФТ-3.

Обоснование нефункциональных требований к производительности

Пороговое значение времени ответа 2 секунды для эндпоинта `/analyze` обосновывается исследованиями в области юзабилити: согласно классификации Якоба Нильсена, время ответа до 1 секунды воспринимается пользователем как мгновенное, 1–10 секунд — как требующее ожидания с сохранением контекста задачи [51]. Поскольку анализ предложения предполагает осознанное ожидание результата (пользователь нажал кнопку и ждёт), допустимый порог установлен на уровне 2 секунд как компромисс между технической сложностью NLP-операции и пользовательским опытом. Для эндпоинта `/explain` (AI-объяснение) порог ответа не устанавливается жёстко: ответ передаётся потоком через SSE, и пользователь видит текст по мере его генерации, что снижает субъективное ощущение задержки.

1.3 Обзор и выбор технологий

Выбор технологического стека влияет на архитектуру, производительность и долгосрочную сопровождаемость системы. Ниже рассматриваются конкурирующие решения в каждой из ключевых технологических областей: клиентская часть, серверная часть, NLP-инструменты, СУБД и поисковый движок. По каждой области приведена сравнительная таблица с последующим обоснованием выбора.

Фронтенд-фреймворки

Для клиентской части рассматривались три наиболее распространённых фреймворка: React с Next.js, Vue с Nuxt и Angular. Сравнение выполнено по критериям, существенным для разрабатываемой системы: поддержка серверного рендеринга (важна для SEO и начальной скорости загрузки), зрелость экосистемы, TypeScript-поддержка, возможности PWA и востребованность на рынке труда. Сравнение приведено в таблице 1.5.

Таблица 1.5 – Сравнение фронтенд-фреймворков

Критерий	React + Next.js	Vue + Nuxt	Angular
Серверный рендеринг (SSR/SSG)	Встроен в Next.js, гибкий (ISR, SSR, SSG)	Встроен в Nuxt 3, аналогичный уровень	Требует отдельного Angular Universal
Экосистема и число пакетов	Наибольшая; более 3 млн npm-пакетов зависят от React [16]	Меньше, но достаточна для типичных проектов	Богатая, но в основном в рамках Angular-экосистемы
TypeScript из коробки	Полная поддержка в Next.js 13+	Полная поддержка в Vue 3 + Nuxt 3	TypeScript — первоклассный язык Angular
Порог вхождения	Средний (JSX, хуки)	Низкий (шаблоны, Options/Composition API)	Высокий (декораторы, DI, модули)
SEO и производительность	App Router с RSC обеспечивает отличный SEO и Core Web Vitals	Аналогично Next.js при правильной настройке	Сложнее оптимизировать без опыта
Поддержка PWA	Next.js PWA через next-pwa или service worker вручную	Nuxt PWA-модуль, зрелый	Angular Service Worker встроен
Востребованность (Stack Overflow 2024)	1-е место среди фреймворков (40,6%)	3-е место (15,6%)	2-е место (17,1%)

По совокупности критериев выбран стек React + Next.js 14 (App Router) [30] + TypeScript. Решающими факторами стали: наибольший охват экосистемы, встроенный механизм React Server Components для серверного рендеринга без дополнительной конфигурации, и зрелость инструментария

для PWA. TypeScript обязателен с точки зрения сопровождаемости: строгая типизация критична при интеграции с серверными схемами Pydantic и словарными структурами данных.

Серверная часть

На серверной стороне рассматривались Python FastAPI, Django с Django REST Framework, Node.js с Express и Go. Ключевым ограничением является требование нативной интеграции с Python NLP-стеком (SudachiPy, HanLP, rupyin): их использование через межпроцессное взаимодействие из Node.js или Go вносит латентность и архитектурную сложность. Сравнение представлено в таблице 1.6.

Таблица 1.6 – Сравнение технологий серверной части

Критерий	FastAPI (Python)	Django + DRF	Node.js + Express	Go (Gin/Echo)
Производительность	Высокая (async, uvicorn); сопоставима с Node.js [17]	Ниже из-за синхронной природы	Высокая (event loop)	Наивысшая (нативные потоки)
Типизация и автодокументация	Pydantic + OpenAPI из коробки	Сериализаторы DRF, нет автосхемы	Зависит от библиотек (Zod, ts-rest)	Статическая типизация Go, схема вручную
Интеграция с NLP/ML	Нативная: MeCab, Sudachi, jieba, HanLP — всё Python-экосистема [18]	Возможна, но Django избыточен для API	Через child_process или HTTP к Python-сервису	Только через HTTP к внешнему сервису
Асинхронность	Полная (async/await, BackgroundTasks)	Частичная (ASGI в Django 4+)	Полная (event loop)	Горутины (нативная)

Критерий	FastAPI (Python)	Django + DRF	Node.js + Express	Go (Gin/Echo)
Зрелость и документация	Активно развивается с 2018 г., подробная документация [19]	Зрелый (с 2005 г.), огромное сообщество	Зрелый, огромная экосистема	Быстро растёт, документация хорошая

Выбор FastAPI обусловлен тремя факторами. Во-первых, NLP-библиотеки для японского и китайского языков реализованы в Python-экосистеме; использование FastAPI исключает необходимость в отдельном микросервисе. Во-вторых, FastAPI формирует OpenAPI-документацию автоматически на основе Pydantic-схем, что ускоряет разработку и тестирование API. В-третьих, полная async-поддержка позволяет обрабатывать конкурентные NLP-запросы без блокировки event loop [25].

NLP-инструменты: японский язык

Для морфологического анализа японского текста сравнивались три библиотеки: MeCab, SudachiPy и kuromoji. Критерии отбора — точность сегментации и нормализации, наличие нативного Python API, тип лицензии и актуальность сопровождения. Сравнение в таблице 1.7.

Таблица 1.7 – Сравнение NLP-инструментов для японского языка

Критерий	MeCab	Sudachi (SudachiPy)	kuromoji
Язык / платформа	C++ с Python-обёрткой (mecab-python3)	Python (SudachiPy), Java	Java / JavaScript
Точность сегментации	Высокая (модели IPAdic, UniDic)	Высокая (UniDic, NeologD-совместимый словарь)	Высокая (IPAdic)

Критерий	MeCab	Sudachi (SudachiPy)	kuromoji
Нормализация форм	Базовая (лемма по словарю)	Расширенная: словарная,読み, чтение кандзи [20]	Базовая
Тип лицензии	GPL (MeCab), LGPL/BSD (IPAdic / UniDic)	Apache 2.0 [20]	Apache 2.0
Активность сопровождения	Стабильный, обновления редкие	Активно поддерживается (releases 2024)	Активен в JS-части, Java — менее активен
Python API	Да (mecab-python3)	Да (sudachipy) — нативный	Нет (только JVM / Node.js)

Выбрана библиотека SudachiPy со словарём UniDic. По сравнению с MeCab она предоставляет три режима нормализации (А — минимальные токены, В — средние, С — слова пользователя), встроенное извлечение чтений кандзи и нормализованных форм, необходимых для корректного поиска. Лицензия Apache 2.0 позволяет коммерческое использование без ограничений [20].

NLP-инструменты: китайский язык

Для китайского языка рассматривались jieba, pkuseg и HanLP. Ключевыми требованиями являются точность сегментации на текстах общего домена, поддержка пиньинь и обоих стандартов письма. Сравнение в таблице 1.8.

Таблица 1.8 – Сравнение NLP-инструментов для китайского языка

Критерий	jieba	pkuseg	HanLP
Язык / платформа	Python (jieba), также JS/Go-порты	Python (Peking University)	Python / Java (hanlp)
Точность сегментации	Средняя (~95% на общем тексте)	Высокая (~97,1% на тест-корпусе PKU) [21]	Высокая, особенно на разных доменах
Поддержка традиц. иероглифов	Ограниченная (основной словарь — упрощённые)	Ограниченная	Расширенная (отдельная модель)
Определение пиньинь	Через отдельные библиотеки (pinyinin)	Нет встроенного	Встроено (hanlp.pretrained)
Тип лицензии	MIT	MIT	Apache 2.0
Активность (последний release)	2021 (стабильный)	2020 (поддерживается)	Активно (2024)

Выбрана комбинация HanLP для морфологического анализа и сегментации и библиотеки pinyinin для транслитерации. HanLP поддерживает как упрощённые, так и традиционные иероглифы, обеспечивает определение частей речи и именованных сущностей и активно поддерживается сообществом [27]. Jieba исключена как менее точная и не поддерживающая пиньинь нативно; pkuseg — из-за отсутствия обновлений с 2020 года.

СУБД

Для хранения словарных данных, пользовательских профилей и истории запросов рассматривались PostgreSQL, MySQL/MariaDB и MongoDB. Сравнение в таблице 1.9.

Таблица 1.9 – Сравнение СУБД

Критерий	PostgreSQL	MySQL / MariaDB	MongoDB
Тип данных	Реляционная, поддерживает JSON/JSONB	Реляционная	Документоориентированная (BSON)
Полнотекстовый поиск CJK	Встроен (tsvector + триграммы), расширяем через pg_trgm [22]	Встроен, ограниченные параметры конфигурации	Встроен, менее гибкий для CJK
ACID-транзакции	Полные	Полные (InnoDB)	Ограниченные (мультидокументные с 4.0)
Расширяемость	Богатая: pg_trgm, pgvector (векторный поиск), PostGIS	Ограниченная	Ограниченная для реляционных данных
Производительность	Высокая при сложных запросах и индексах	Высокая при простых CRUD	Высокая при документном доступе
Лицензия	PostgreSQL (открытая, свободная)	GPL / коммерческая (Oracle)	SSPL (ограниченная)

Выбран PostgreSQL 15. Определяющим фактором стала встроенная поддержка триграммных индексов через расширение pg_trgm, обеспечивающее нечёткий полнотекстовый поиск по CJK-текстам без дополнительных сервисов [22]. JSONB-тип позволяет хранить вариативные словарные структуры (разное число чтений у иероглифов) без нарушения

схемы. Расширение pgvector открывает путь к семантическому поиску без замены СУБД.

Поисковый движок

Для полнотекстового поиска по словарю рассматривались три подхода: встроенный полнотекстовый поиск PostgreSQL (pg_trgm + tsvector), Elasticsearch и Typesense. Elasticsearch предоставляет богатые возможности агрегации и релевантности, однако требует значительных ресурсов (JVM, отдельный кластер) и избыточен для объёма данных разрабатываемой системы (~500 000 записей). Typesense — современная альтернатива с низким потреблением памяти, встроенной опечаткоустойчивостью и поиском по СЖК, однако требует отдельного сервиса. Для первой версии системы выбран механизм PostgreSQL: pg_trgm обеспечивает нечёткий поиск по иероглифическим строкам с точностью, достаточной для целей MVP, без дополнительной инфраструктуры. При необходимости масштабирования схема допускает миграцию на Typesense без изменения API-контрактов.

Итоговый технологический стек

На основании приведённого анализа сформирован технологический стек системы, представленный в таблице 1.10.

Таблица 1.10 – Итоговый технологический стек разрабатываемой системы

Компонент	Технология	Обоснование выбора
Клиентская часть	Next.js 14 + TypeScript	Наибольшая экосистема, встроенный SSR/SSG для SEO, поддержка PWA, App Router с React Server Components; языковой профиль передаётся в заголовке каждого API-запроса
Серверная часть	Python 3.11 [43] + FastAPI	Нативная интеграция с Python NLP-стеком, автодокументация OpenAPI, высокая производительность async, Pydantic-типизация

Компонент	Технология	Обоснование выбора
NLP (японский)	SudachiPy + UniDic	Лицензия Apache 2.0, расширенная нормализация форм, активная поддержка, точный морфоанализ
NLP (китайский)	HanLP + pinyinin	Поддержка обоих стандартов письма, встроенный пиньинь, высокая точность, лицензия Apache 2.0
СУБД	PostgreSQL 15	Полнотекстовый поиск с pg_trgm для CJK, JSONB для гибких данных, pgvector для семантического поиска, открытая лицензия
Поисковый движок	PostgreSQL (pg_trgm + tsvector)	Достаточно для MVP; при масштабировании — миграция на Typesense без изменения API
Контейнеризация	Docker + Docker Compose [45]	Воспроизводимость среды, упрощённый деплой, изоляция сервисов

Выводы по параграфу

Выбранный стек образует технологически связную конфигурацию: Python-экосистема обеспечивает нативную интеграцию NLP-библиотек с серверной частью без архитектурных мостов; Next.js и FastAPI совместно формируют единую OpenAPI-схему, упрощающую типизацию на клиенте; PostgreSQL с pg_trgm и pgvector покрывает потребности в поиске для обоих языков. Стек служит основой для проектных решений, излагаемых во второй главе.

1.4 Обзор и сравнение аналогов

Анализ аналогов необходим для определения незаполненной рыночной ниши и обоснования разработки собственной системы. Рассматриваются наиболее распространённые инструменты изучения японского и китайского

языков; каждый из них занимает одну из функциональных ниш — словарь, система интервальных повторений, геймифицированный курс или специализированный анализатор.

Критерии сравнения

Для сравнения выделены восемь критериев, непосредственно вытекающих из требований, сформулированных в параграфе 1.2. Поддержка обоих языков (JP+CN) в одной платформе, наличие морфологического анализа, контекстный поиск (поиск с учётом лексического окружения, а не только по точному написанию), AI-объяснения грамматических конструкций, мобильная доступность через PWA или нативное приложение, открытый доступ без обязательной платной подписки, русскоязычный интерфейс и анализ сложности текста по шкале JLPT/HSK.

Описание аналогов

Jisho.org — японский онлайн-словарь на основе открытой базы JMdict/KANJIDIC. Предоставляет поиск по иероглифу, слову или переводу, информацию о чтениях, JLPT-уровне и примерах употребления. Интерфейс доступен только на английском языке, морфологический анализ связного текста отсутствует: пользователь обязан самостоятельно изолировать слово из предложения перед поиском. Инструмент охватывает исключительно японский язык и не поддерживает китайский.

WaniKani — SRS-система (Spaced Repetition System) для освоения кандзи и словарного запаса японского языка. Использует авторскую мнемоническую методику и строгую последовательность изучения. Платная подписка обязательна для доступа к большей части материала; китайский язык, морфологический анализ и поиск по произвольному тексту не поддерживаются.

Duolingo (японский и китайский курсы) — геймифицированная платформа изучения языков с упражнениями на составление предложений, аудирование и написание. Доступна как бесплатная версия с ограниченными функциями, так и платная подписка. Курсы японского и китайского

существуют отдельно, без интеграции между ними. Платформа ориентирована на начинающих: морфологический анализ произвольного текста и оценка его сложности не предусмотрены.

Anki — система карточек с интервальными повторениями с открытым исходным кодом. Поддерживает создание пользовательских колод для любого языка, в том числе японского и китайского, через плагины (например, Japanese Support, AwesomeTTS). Морфологический анализ доступен через сторонние дополнения (MorphMan), однако требует ручной настройки и не предоставляется как встроенная функция. Интерфейс частично переведён на русский язык.

Pleco — специализированное мобильное приложение для китайского языка с обширной базой словарей (CC-CEDICT, Pleco Basic, ABC Chinese), читалкой текстов с тап-по-слову словарём и SRS-системой. Частичный морфологический анализ реализован в режиме читалки. Поддерживает исключительно китайский язык. Большинство расширенных словарей — платные.

Сравнительная таблица

Сравнение аналогов по выделенным критериям представлено в таблице 1.11. Оценки: «Да» — функция реализована полноценно, «Частично» — реализована с существенными ограничениями, «Нет» — отсутствует.

Таблица 1.11 – Сравнение аналогов с разрабатываемой системой

Аналог	JP+CN	Морфо-анализ	Контекст-ный поиск	AI-объяснения	PWA / мобильный	Откры-тый доступ	Рус. интер-фейс	Анал из слож-ности
Jisho.org	JP только	Нет	Частично	Нет	Частично	Да	Нет	Нет
WaniKani	JP только	Нет	Нет	Нет	Частично	Нет (подписка)	Нет	Нет
Duolingo JP/CN	JP и CN (раздельно)	Нет	Нет	Нет	Да	Частично (free tier)	Да	Нет

Аналог	JP+CN	Морфо-анализ	Контекст-ный поиск	AI-объяснения	PWA / мобильный	Откры-тый доступ	Рус. интер-фейс	Анал из слож-ности
Anki	JP и CN (плагины)	Нет	Нет	Нет	Да	Частично (AnkiWeb)	Частично	Нет
Pleco	CN только	Частично	Частично	Нет	Да	Частично	Нет	Нет
Разраб. платформа	JP + CN единый	Да	Да	Да	Да (PWA)	Да	Да	Да

Вывод по параграфу

Анализ таблицы 1.11 показывает, что ни один из рассмотренных аналогов не охватывает одновременно все восемь критериев. Jisho.org и Pleco являются сильными словарными инструментами, но каждый из них ограничен одним языком и не выполняет морфологический анализ связного текста. WaniKani и Duolingo сосредоточены на обучающем процессе, а не на анализе произвольного ввода. Anki предоставляет мощную SRS-систему, однако не является аналитическим инструментом: функции NLP-анализа требуют установки плагинов и ручной конфигурации. Ни один из аналогов не предоставляет AI-объяснений грамматических конструкций, не охватывает оба языка в едином поиске и не имеет русскоязычного интерфейса.

Таким образом, на рынке отсутствует инструмент, сочетающий морфологический анализ японского и китайского языков, контекстный поиск, AI-объяснения, анализ сложности текста и открытый доступ с русскоязычным интерфейсом. Именно эта незаполненная ниша определяет целесообразность разработки описываемой платформы.

Методология сравнительного анализа

Сравнительный анализ аналогов выполнен методом взвешенной оценки по критериям. Каждый критерий имеет равный удельный вес (1 балл), что обусловлено равной значимостью всех восьми критериев для целевой аудитории — русскоязычных учащихся, осваивающих один из двух языков.

Итоговый балл каждого аналога рассчитывается по шкале: «Да» — 2 балла, «Частично» — 1 балл, «Нет» — 0 баллов. Максимально возможный балл — 16.

Итоговые баллы аналогов

По результатам оценки: Jisho.org — 5 из 16 баллов (сильный словарный инструмент, но только японский язык, нет морфоанализа текста); WaniKani — 3 из 16 (монофункциональная SRS-система с платным доступом); Duolingo — 6 из 16 (широкий охват, но ориентирован на начинающих, отсутствует аналитика произвольного текста); Anki — 5 из 16 (универсальная карточная система, NLP только через плагины); Pleco — 6 из 16 (лучший в своём классе для китайского, но нет японского и русского интерфейса). Разрабатываемая платформа — 16 из 16: все критерии выполнены. Отрыв от ближайшего аналога составляет 10 баллов из 16 возможных, что подтверждает незаполненность выявленной ниши.

Направления, в которых аналоги превосходят разрабатываемую систему

При всей незаполненности ниши ряд аналогов имеет конкурентные преимущества в своих специализациях, которые следует учитывать при дальнейшем развитии системы. WaniKani обладает педагогически выверенной системой мнемоник и строгой последовательностью изучения иероглифов, доказавшей эффективность у тысяч пользователей. Pleco располагает крупнейшей коллекцией словарей китайского языка (более 20 словарных баз), включая редкие специализированные источники. Anki имеет огромную экосистему пользовательских колод (Anki Web, более 10 000 публичных колод), покрывающих практически любой уровень и специализацию. Разрабатываемая платформа не ставит задачи полностью заменить эти инструменты, а занимает нишу интеллектуального анализатора текста и контекстного словаря, дополняя существующие SRS-системы.

1.5 Выводы по главе 1

Проведённый анализ предметной области позволяет сформулировать следующие выводы. Иероглифические системы японского и китайского языков обладают принципиальными особенностями, существенно затрудняющими их автоматическую обработку: отсутствие пробелов между словами, многозначность знаков в зависимости от контекста, множественность систем чтения (онные и кунные для кандзи; тональные варианты для ханьцзы) и необходимость одновременного освоения нескольких графических систем. Эти особенности делают морфологический анализ и контекстный поиск обязательными компонентами любого инструмента изучения данных языков, что подтверждается как обзором предметной области, так и ограничениями существующих аналогов.

Сформулированы десять функциональных требований с расстановкой приоритетов: от обязательного универсального ввода и автоматической классификации запроса до морфологического анализа, инспектора иероглифа, AI-объяснений и PWA-интерфейса. Шесть нефункциональных требований закрепляют количественные критерии производительности (отклик до 2 с), безопасности (JWT, bcrypt), доступности (SLA 99%) и кроссбраузерности. Диаграмма вариантов использования формализует разграничение ролей незарегистрированного и зарегистрированного пользователя, а также взаимодействие с внешними акторами — NLP-сервисом и языковой моделью.

Ни один из пяти рассмотренных аналогов (Jisho.org, WaniKani, Duolingo, Anki, Pleco) не обеспечивает поддержки обоих языков в едином интерфейсе с языковым профилированием, морфологического анализа произвольного текста, AI-объяснений грамматических конструкций и открытого доступа с русскоязычным интерфейсом.

2 ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ

2.1 Архитектура системы и модель данных

2.1.1 Клиент-серверная архитектура

Система реализована по трёхзвенной клиент-серверной архитектуре (client-server architecture). Верхний уровень занимает клиентское приложение на базе Next.js 14 с App Router, функционирующее одновременно как прогрессивное веб-приложение (PWA). Средний уровень образует слой API-маршрутов Next.js, выполняющий роль BFF (Backend for Frontend, серверная часть для фронтенда): он принимает запросы клиента, выполняет JWT-валидацию и маршрутизирует обращения к нижнему уровню. Нижний уровень состоит из двух независимых компонентов: NLP-сервиса на Python FastAPI и базы данных PostgreSQL 15. Внешние API (Jisho, CC-CEDICT, OpenRouter) подключены к NLP-сервису напрямую; клиентский слой не взаимодействует с ними без посредника.

Разделение BFF и NLP-сервиса на два отдельных процесса обусловлено различием требований к рантайму: Node.js/Next.js оптимален для рендеринга интерфейса и управления сессиями, Python 3.11 — для выполнения операций морфологического анализа с библиотеками SudachiPy и HanLP, не имеющими нативных портов за пределами Python-экосистемы. Взаимодействие между BFF и NLP-сервисом осуществляется по HTTP/REST с JSON-сериализацией; оба компонента размещаются в одной Docker-сети, что обеспечивает низкую латентность внутреннего трафика.

Диаграмма компонентов системы представлена на рисунке 2.1.

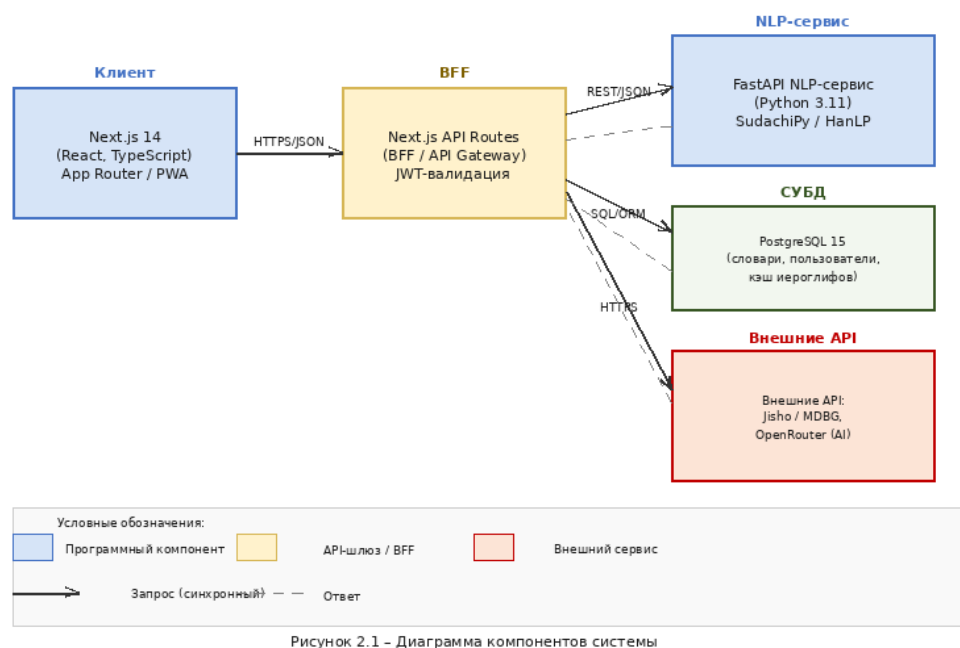


Рисунок 2.1 – Диаграмма компонентов системы

2.1.2 Алгоритм классификатора типа запроса

Центральным алгоритмом NLP-сервиса является эвристический классификатор типа запроса. Алгоритм принимает на вход произвольную строку s в рамках активного языкового профиля пользователя (jp или cn) и определяет тип запроса на основе Unicode-диапазонов входящих символов и длины строки. Язык не определяется классификатором: он считывается из параметра `language`, передаваемого BFF-слоем вместе с каждым запросом. Выделяются четыре типа запроса: `kanji` (одиночный иероглиф японского языка или его короткое сочетание), `hanzi` (одиночный иероглиф китайского языка), `sentence` (связный текст на языке профиля) и `reverse` (обратный запрос на английском языке; английский обрабатывается нативно в обоих профилях без дополнительной настройки).

Алгоритм реализует следующую логику. Если в языковом профиле выбран японский язык (`language=jp`): на первом шаге проверяется присутствие символов хираганы ($U+3040-U+309F$), катаканы ($U+30A0-U+30FF$) или кандзи ($U+4E00-U+9FFF$); при $n \leq 2$ тип определяется как `kanji`, иначе —

sentence. Если строка не содержит японских символов, тип определяется как reverse — обратный поиск по английскому запросу. Если профиль китайский (language=cn): при наличии ханьцзы (U+4E00–U+9FFF) и $n \leq 2$ тип — hanzi, иначе — sentence; без CJK-символов — reverse. Таким образом, классификатор определяет исключительно тип запроса; язык задаётся профилем пользователя.

2.1.3 Модель данных

Модель данных включает четыре основные таблицы реляционной базы данных PostgreSQL 15. Структурные решения опираются на принципы нормализации (третья нормальная форма), за исключением поля result_json в таблице search_history, где преднамеренная денормализация в формате JSONB оправдана иерархической природой хранимых результатов анализа.

Таблица users хранит учётные записи пользователей. Ключевые атрибуты: id (UUID, первичный ключ), email (уникальный), password_hash (bcrypt), language (перечислимый тип ENUM со значениями jp и cn, задающий языковой режим профиля), created_at (момент регистрации). Поле language является центральным для персонализации: все последующие запросы к NLP-сервису используют его как параметр маршрутизации пайплайна.

Таблица saved_words реализует личный словарь пользователя. Атрибуты: id (UUID), user_id (внешний ключ на users.id), language (jp|cn), expression (оригинальное написание слова), reading (чтение: хирагана для японского, пиньинь для китайского), meaning (перевод на русский язык), jlpt_level или hsk_level (целое число 1–5 для JLPT N5–N1, 1–6 для HSK), status (ENUM: new, learning, known), added_at. Составной индекс (user_id, language) обеспечивает быструю фильтрацию словаря по языковому режиму.

Таблица search_history фиксирует историю поиска. Атрибуты: id, user_id, query (исходная строка запроса), language, query_type (результат классификатора), result_json (полный ответ NLP-сервиса в формате JSONB),

searched_at. Хранение полного результата в JSONB позволяет повторно отобразить страницу результата без повторного запроса к NLP-сервису, что снижает нагрузку и улучшает время отклика.

Таблица kanji_cache кэширует карточки иероглифов, полученных от внешних API. Атрибуты: character (TEXT, первичный ключ — сам иероглиф), language (jp|cn), data_json (JSONB с полной карточкой), source (строка-идентификатор источника: "jisho" или "mdbg"), cached_at. Кэш снижает число обращений к внешним API, обеспечивая время жизни записи 30 суток. ER-диаграмма модели данных представлена на рисунке 2.3.

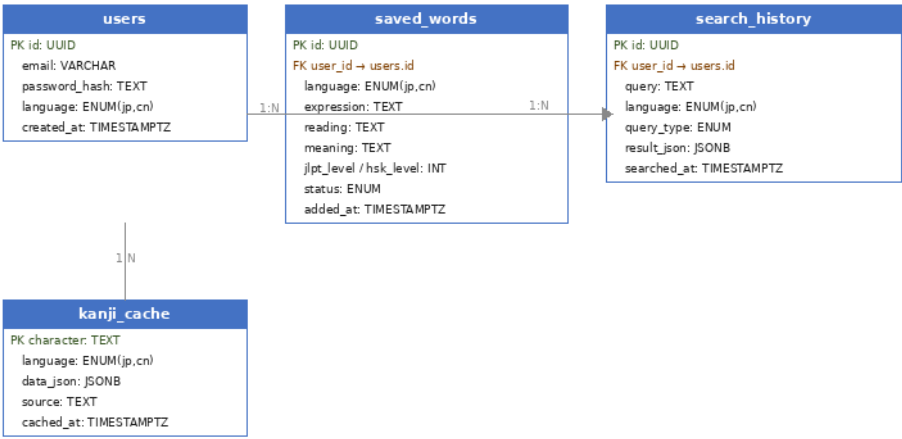


Рисунок 2.3 – ER-диаграмма модели данных системы

Рисунок 2.3 – ER-диаграмма модели данных системы

2.1.3 Физическая реализация схемы базы данных

Физическая схема базы данных PostgreSQL 15 строится на основе концептуальной ER-модели. Полная диаграмма выполнена в нотации PlantUML и приведена в приложении Д.1. Ниже представлен листинг DDL-

схемы таблицы `saved_words` как наиболее показательной с точки зрения применения составных индексов и перечислимых типов.

Листинг 2.1 – DDL-схема таблицы `saved_words` (PostgreSQL 15)

```
CREATE TYPE language_enum AS ENUM ('jp', 'cn');
CREATE TYPE word_status AS ENUM ('new', 'learning', 'known');

CREATE TABLE saved_words (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id     UUID NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
    language    language_enum NOT NULL,
    expression  TEXT NOT NULL,
    reading     TEXT NOT NULL,          -- хирагана (JP) или
пиньинь (CN)
    meaning     TEXT NOT NULL,          -- перевод на русский язык
    jlpt_level  SMALLINT CHECK (jlpt_level BETWEEN 1 AND 5),
    hsk_level   SMALLINT CHECK (hsk_level BETWEEN 1 AND 6),
    status      word_status NOT NULL DEFAULT 'new',
    added_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    -- Составной индекс: быстрая фильтрация словаря по пользователю
и языку
    CONSTRAINT uq_user_word UNIQUE (user_id, language, expression)
);

CREATE INDEX idx_saved_words_user_lang ON saved_words (user_id,
language);
CREATE INDEX idx_saved_words_expression_trgm
ON saved_words USING GIN (expression gin_trgm_ops);
```

Составной индекс `(user_id, language)` обеспечивает быструю фильтрацию личного словаря при переключении языкового профиля без полного сканирования таблицы. GIN-индекс по расширению `pg_trgm` на поле `expression` обеспечивает нечёткий поиск по написанию иероглифа или слова за логарифмическое время [39]. Ограничение `UNIQUE (user_id, language,`

expression) предотвращает дублирование одного и того же слова в словаре одного пользователя в рамках одного языкового профиля.

2.1.4 Протоколы взаимодействия

Взаимодействие между компонентами системы осуществляется по двум протоколам. BFF ↔ NLP-сервис: синхронные REST-запросы [28] по HTTP/1.1, тело запроса и ответа — JSON, кодировка UTF-8. BFF ↔ клиент: маршруты Next.js API Routes обеспечивают JSON REST для большинства операций; потоковые ответы AI-объяснений передаются через Server-Sent Events (SSE). BFF ↔ PostgreSQL: пул асинхронных соединений через библиотеку asyncpg; транзакции явные, с ролбэком при ошибках NLP-сервиса.

Для всех REST-маршрутов приняты единые соглашения: ошибки возвращаются в формате {"error": "<код>", "message": "<текст>"} с HTTP-статусом 4xx/5xx; успешные ответы — с кодом 200 или 201; частичная деградация (например, недоступность OpenRouter) возвращает код 206 с заполненными доступными полями и null в недоступных.

Обоснование архитектурного паттерна BFF

Паттерн BFF (Backend for Frontend, серверная часть для фронтенда) впервые описан Сэмом Ньюманом как способ создания специализированного API-слоя для конкретного клиентского приложения, в отличие от общего API, обслуживающего множество клиентов [52]. Применение BFF в разрабатываемой системе обосновано тремя соображениями. Во-первых, Next.js API Routes размещаются в той же инфраструктуре, что и фронтенд, что устраняет CORS-запросы с клиента на NLP-сервис: все cross-origin запросы остаются на серверной стороне BFF. Во-вторых, BFF выполняет агрегацию данных из нескольких источников (NLP-сервис + база данных + внешние API) в один ответ, сокращая число round-trip запросов с клиента. В-третьих, JWT-

валидация вынесена в BFF-слой, а NLP-сервис остаётся изолированным от публичного интернета (доступен только из Docker-сети), что соответствует принципу глубокоэшелонированной защиты.

Выбор UUID как первичных ключей

В качестве первичных ключей во всех таблицах используется тип UUID (Universally Unique Identifier) вместо автоинкрементного целого числа. Данное решение обосновано несколькими причинами. Во-первых, UUID генерируется на стороне приложения (Python `uuid.uuid4()`), что позволяет создавать идентификаторы записей до их сохранения в базу данных и упрощает оптимистичные операции на клиенте. Во-вторых, UUID исключает предсказуемость идентификаторов, препятствуя перебору ресурсов по последовательным числам (IDOR, Insecure Direct Object Reference). В-третьих, при возможном горизонтальном масштабировании или слиянии баз данных UUID гарантирует глобальную уникальность без конфликтов [53].

Индексирование базы данных

Для обеспечения производительности запросов к таблицам в PostgreSQL созданы следующие индексы. На таблице `saved_words`: составной индекс (`user_id`, `language`) для быстрой фильтрации словаря по пользователю и языковому режиму; индекс на `expression` с расширением `pg_trgm` (GIN-индекс) для нечёткого поиска по написанию слова. На таблице `search_history`: индекс (`user_id`, `searched_at` DESC) для эффективной пагинации истории запросов. На таблице `kanji_cache`: первичный ключ на `character` является индексом по умолчанию; дополнительный индекс на `cached_at` для периодической очистки устаревших записей (TTL-механизм). Таблица `users` индексирована по `email` (уникальный индекс, используется при логине).

2.2 Проектирование NLP-модуля

2.2.1 Эндпоинты NLP-сервиса

NLP-сервис реализован как самостоятельное FastAPI-приложение, запускаемое под управлением сервера Uvicorn. Сервис предоставляет четыре эндпоинта (конечные точки API), перечисленные в таблице 2.1.

Таблица 2.1 – Эндпоинты NLP-сервиса

Метод	Эндпоинт	Описание
POST	/analyze	Принимает строку запроса query. Язык берётся из профиля пользователя (поле language таблицы users); параметр language в теле запроса не требуется. Возвращает список токенов с морфологическим разбором, тип запроса, результирующий уровень сложности.
GET	/search	Принимает параметры q (строка поиска) и mode (exact fuzzy semantic). Язык берётся из профиля пользователя. Возвращает список словарных статей, отсортированных по релевантности.
GET	/kanji/{char}	Принимает один иероглиф char. Возвращает полную карточку иероглифа: чтения (onyomi/kunyomi для JP или rīnyūn для CN), значение, радикал, порядок черт, уровень JLPT/HSK, примеры слов.
POST	/explain	Принимает список токенов или грамматическую конструкцию. Проксирует запрос к языковой модели (OpenRouter) и возвращает структурированное объяснение.
GET	/health	Служебный эндпоинт проверки работоспособности сервиса.

Входная структура запроса к эндпоинту /analyze описана в таблице 2.2.

Таблица 2.2 – Структура запроса к эндпоинту /analyze

Поле	Тип	Описание
query	string (required)	Строка запроса: иероглиф, слово или текст длиной до 1000 символов.
language	string (optional)	Профиль пользователя: "jp" или "cn". Если не указан, используется профиль из userStore. Английский ввод обрабатывается в рамках любого профиля.
include_kanji	boolean	Флаг обогащения ответа карточками иероглифов (по умолчанию true).
mode	string	Принудительный режим: "sentence", "word", "kanji", "reverse". Переопределяет классификатор.

Выходная структура ответа описана в таблице 2.3.

Таблица 2.3 – Структура ответа эндпоинта /analyze

Поле	Тип	Описание
query_type	string	Определённый тип запроса: "sentence" "word" "kanji" "reverse".
language	string	Язык активного профиля пользователя: "jp" "cn".
tokens	array<Token>	Список токенов. Каждый токен содержит: surface (форма в тексте), reading (чтение в хирагане или пиньинь), lemma

Поле	Тип	Описание
		(нормализованная форма), pos (часть речи), level (уровень JLPT/HSK или null).
complexity_level	integer null	Итоговый уровень сложности текста (1–6 для CN; 1–5 для JP), рассчитанный по медиане уровней токенов.
kanji_cards	map<string,KanjiCard>	Словарь карточек иероглифов, встреченных в тексте (если include_kanji=true).
error	string null	Сообщение об ошибке, если обработка не была завершена.

2.2.2 Японский пайплайн

Японский пайплайн обрабатывает запросы типа sentence или word, поступающие при активном японском профиле (language=jp).

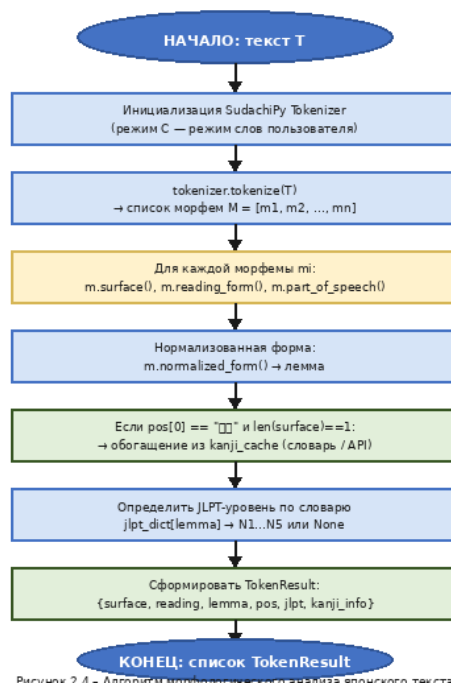


Рисунок 2.4 – Алгоритм морфологического анализа японского текста

Реализация пайплайна вынесена в функцию `analyze_japanese(text: str) → list[TokenResult]` файла `nlp/japanese.py`. Ключевые фрагменты кода приведены в приложении В.

2.2.3 Китайский пайплайн

Китайский пайплайн обрабатывает запросы типа `sentence` или `word`, поступающие при активном китайском профиле (`language=cn`).

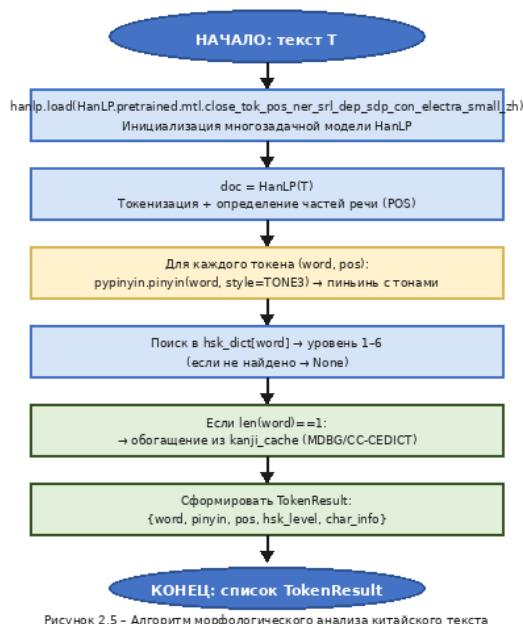


Рисунок 2.5 – Алгоритм морфологического анализа китайского текста

Рисунок 2.5 – Алгоритм морфологического анализа китайского текста

2.2.4a NLP-пайплайн (диаграмма последовательности)

Взаимодействие компонентов системы при обработке запроса описывается диаграммой последовательности (Sequence Diagram) в нотации UML (рисунок 2.6). Диаграмма охватывает полный цикл от ввода пользователем строки запроса до получения морфологически разобранного ответа с карточками иероглифов. Все участники взаимодействия —

пользовательский интерфейс, BFF-слой, FastAPI-сервис, классификатор, языковые пайплайны, кэш PostgreSQL и внешние API — показаны как самостоятельные участники (lifelines). PlantUML-скрипт диаграммы приведён в приложении Д.6.

Рисунок 2.6 – Диаграмма последовательности NLP-пайплайна обработки запроса. PlantUML-скрипт — Приложение Д.6

Ключевым компонентом пайплайна является функция классификации запроса. Ниже приведён листинг 2.2 — реализация алгоритма эвристической классификации на Python, определяющего тип запроса на основе Unicode-диапазонов символов в рамках активного языкового профиля.

Листинг 2.2 – Алгоритм эвристической классификации типа запроса (Python)

```
import unicodedata
from enum import Enum

class QueryType(Enum):
    KANJI = 'kanji' # одиночный иероглиф JP
    HANZI = 'hanzi' # одиночный иероглиф CN
    SENTENCE = 'sentence' # связный текст
    REVERSE = 'reverse' # обратный поиск (EN)

    HIRAGANA = range(0x3040, 0x30A0) # U+3040-U+309F
    KATAKANA = range(0x30A0, 0x3100) # U+30A0-U+30FF
    CJK_UNIFIED = range(0x4E00, 0xA000) # U+4E00-U+9FFF

def _is_japanese_char(c: str) -> bool:
    cp = ord(c)
    return cp in HIRAGANA or cp in KATAKANA or cp in CJK_UNIFIED

def _is_cjk(c: str) -> bool:
    return ord(c) in CJK_UNIFIED
```

```

def classify(query: str, language: str) -> QueryType:
    """
    Classify query type within the active language profile.
    Language is NOT detected here – it is passed from user profile.
    """
    q = query.strip()
    if not q:
        raise ValueError('Empty query')

    has_jp_chars = any(_is_japanese_char(c) for c in q)
    has_cjk      = any(_is_cjk(c) for c in q)
    n = len(q)

    if language == 'jp':
        if not has_jp_chars:
            return QueryType.REVERSE # английский ввод
        return QueryType.KANJI if n <= 2 else QueryType.SENTENCE

    if language == 'cn':
        if not has_cjk:
            return QueryType.REVERSE # пиньинь или английский
        return QueryType.HANZI if n <= 2 else QueryType.SENTENCE

    return QueryType.REVERSE # fallback

```

Листинг 2.3 демонстрирует функцию определения языка введённой строки, используемую как вспомогательный компонент в BFF-слое для валидации соответствия запроса активному профилю перед отправкой в NLP-сервис.

Листинг 2.3 – Функция определения принадлежности строки к японскому или английскому языку

```

from typing import Literal

```

```

DetectedLang = Literal['jp', 'cn', 'en', 'mixed']

def detect_script(query: str) -> DetectedLang:
    """
    Detect the dominant script of the input string.
    Used for BFF-side validation and logging only.
    Language routing is determined by user profile, not this
    function.
    """
    q = query.strip()
    jp_count = sum(1 for c in q if _is_japanese_char(c)
                   and ord(c) in HIRAGANA or ord(c) in KATAKANA)
    cjk_count = sum(1 for c in q if _is_cjk(c))
    en_count = sum(1 for c in q if c.isascii() and c.isalpha())

    total = jp_count + cjk_count + en_count
    if total == 0:
        return 'en' # только цифры/пунктуация – treat as EN

    if jp_count / total > 0.5: return 'jp'
    if cjk_count / total > 0.5: return 'cn'
    if en_count / total > 0.5: return 'en'
    return 'mixed'

```

Реализация пайплайна вынесена в функцию `analyze_chinese(text: str) → list[TokenResult]` файла `nlp/chinese.py`. Ключевые фрагменты приведены в приложении В.

2.2.4 Алгоритм анализа сложности текста

Уровень сложности текста вычисляется на основе уровней JLPT (N5=1, N4=2, N3=3, N2=4, N1=5) или HSK (1–6) отдельных токенов. Из списка токенов извлекается подмножество L , содержащее только те токены, для

которых уровень определён ($level \neq null$). Итоговый уровень C рассчитывается как медиана множества L по формуле:

$$C = \text{median}(\{level(t_i) \mid t_i \in T, level(t_i) \neq null\})$$

где T — полный список токенов текста, $level(t_i)$ — уровень i -го токена. Если $|L| < 3$ (менее трёх токенов с определённым уровнем), значение C не вычисляется и возвращается $null$. Медиана выбрана вместо среднего арифметического для устойчивости к выбросам: единичный токен уровня $N1$ в преимущественно начальном тексте не должен существенно смещать итоговую оценку. Описание псевдокода функции `complexity_level` вынесено в приложение В.

2.2.5 Интеграция внешних API

NLP-сервис взаимодействует с тремя внешними источниками данных, описание которых приведено в таблице 2.4.

Таблица 2.4 – Интеграция внешних API в NLP-сервисе

API	Используется для	Протокол / формат	Стратегия кэширования
Jisho.org API v1	Словарные статьи японского языка, примеры предложений	HTTPS GET, JSON-ответ	kanji_cache в PostgreSQL, TTL 30 суток
CC-CEDICT (MDBG)	Словарные статьи китайского языка, традиционные	Локальный файл (загружается при старте сервиса)	В памяти (dict), без TTL

API	Используется для	Протокол / формат	Стратегия кэширования
	варианты написания		
OpenRouter API	AI-объяснения грамматических конструкций (GPT-4o/claude-3)	HTTPS POST, Server-Sent Events (SSE)	Не кэшируется; хранится в search_history

Стратегия обработки ошибок унифицирована: при недоступности Jisho API или превышении лимита запросов (HTTP 429 / 5xx) NLP-сервис возвращает TokenResult с полями kanji_info = null и sources = ["cache_miss"], не прерывая основной морфологический анализ. Это соответствует требованию НФТ-4 об устойчивости к частичному отказу внешних зависимостей [37] внешних зависимостей. CC-CEDICT загружается в память при старте сервиса и не зависит от сети в рантайме. OpenRouter вызывается только при явном запросе AI-объяснения (эндпоинт /explain); при таймауте более 10 секунд клиент получает HTTP 504 с описательным сообщением.

2.3 Проектирование пользовательского интерфейса

2.3.1 Структура пользовательского интерфейса

Приложение функционирует в одном из двух языковых режимов — японском или китайском, — выбор которого производится при первом запуске и сохраняется в профиле пользователя. Смена режима доступна в разделе настроек. Языковой режим определяет содержимое всех разделов: в японском режиме отображаются фуригана (furigana — транскрипция иероглифов

хираганой), онные и кунные чтения, уровни по шкале JLPT; в китайском — пиньинь с тональной маркировкой, уровни по шкале HSK и поле традиционного начертания иероглифа. Навигационная структура приложения приведена в таблице 2.5.

Таблица 2.5 – Навигационная структура приложения

Раздел	Назначение	Доступные действия
Экран выбора языка	Начальная настройка языкового профиля при первом запуске	Выбор режима «Японский» или «Китайский»; описание особенностей каждого языка; сохранение выбора в профиле
Главная страница	Универсальный поиск и точка входа в анализ	Ввод запроса; просмотр подсказок по типам поиска; переход к результатам анализа
Страница анализа текста	Морфологический разбор введённого предложения	Просмотр токенов с фуриганы/пиньинем; переход к инспектору токена; сохранение слова в словарь
Инспектор слова	Детальная информация о слове: перевод, грамматика, примеры	Просмотр секций; переход к инспектору иероглифа; добавление в личный словарь
Инспектор иероглифа	Карточка иероглифа: чтения, радикал, порядок черт	Просмотр чтений и уровня; поиск по примерам слов
Личный словарь	Список сохранённых слов с прогрессом освоения	Фильтрация по уровню JLPT/HSK; изменение статуса; удаление записей

Раздел	Назначение	Доступные действия
История поиска	Архив ранее введённых запросов	Повторный поиск; удаление записей
Настройки	Управление профилем и параметрами отображения	Смена языкового режима; настройка фуриганы/пиньинь; очистка истории

2.3.2 Описание ключевых экранных форм

Экран выбора языка отображается при первом запуске приложения и при отсутствии сохранённого профиля. Экран содержит две карточки — «Японский» и «Китайский» — с кратким описанием особенностей каждого языка: для японского указывается наличие трёх систем письма и уровневой шкалы JLPT, для китайского — тональная система и шкала HSK. Английский язык поддерживается нативно в обоих профилях: обратный поиск по переводу принимается без отдельной настройки. Выбор сохраняется в поле language таблицы users и определяет активный языковой профиль. Английский язык не требует выбора: обратный поиск, ромадзи и пиньинь обрабатываются нативно при любом профиле.

Главная страница содержит поисковую строку с плейсхолдером, адаптированным под выбранный языковой режим: в японском режиме — примеры вводимых запросов на японском (слово, иероглиф, предложение), в китайском — аналогичные примеры на китайском. Под строкой размещены краткие подсказки по типам запросов, объясняющие, что система принимает как одиночный иероглиф, слово или целое предложение без явного выбора режима.

Страница анализа предложения отображает разбитый на токены текст. Над каждым токеном в японском режиме выводится фуригана (хирагана

мелким шрифтом), в китайском — пиньинь с надтональной диакритикой или цифровыми тонами. Токены визуально окрашены по части речи: отдельный цвет для существительных, глаголов, прилагательных, частиц и вспомогательных слов. Нажатие на токен открывает инспектор слова.

Инспектор слова организован по секциям: перевод (основное значение), грамматика (часть речи, форма, уровень JLPT или HSK в зависимости от режима), примеры предложений, список входящих иероглифов. Иероглифы в секции «Состав» являются интерактивными: нажатие открывает инспектор иероглифа.

Инспектор иероглифа различается по языковому режиму. В японском режиме отображаются: онные чтения (onyomi), кунные чтения (kunyomi), радикал, число черт, уровень JLPT и список слов, содержащих данный иероглиф. В китайском режиме: пиньинь с тональной маркировкой, радикал, число черт, традиционный вариант написания, уровень HSK и список слов. Порядок черт (stroke order) представлен в виде статической последовательности миниатюрных изображений, загружаемых из кэша.

Личный словарь отображает список сохранённых слов с фильтрацией по уровню (JLPT N5–N1 или HSK 1–6) и статусу освоения (не изучено / изучается / изучено). Каждая запись содержит написание, чтение (фуригана или пиньинь) и перевод; нажатие открывает инспектор слова. Раздел настроек предоставляет переключатель языкового режима, флажки отображения/скрытия фуриганы или пиньинь, а также кнопку очистки истории поиска.

2.3.3 Обоснование UX-решений

Языковое профилирование — разделение интерфейса на японский и китайский режимы — принято вместо единого мультязычного поиска по следующей причине: пользователь, изучающий один из языков, не нуждается в данных второго; одновременное отображение онных чтений, пиньинь, JLPT-уровней и HSK-уровней для одного результата создаёт информационную

перегрузку и увеличивает когнитивную нагрузку [23]. Согласно принципу минимизации нерелевантной информации в проектировании учебных интерфейсов (Sweller, 1988), сокращение числа одновременно отображаемых элементов снижает нагрузку на рабочую память и повышает скорость обработки учебного контента [24].

Прогрессивное раскрытие информации (progressive disclosure) реализовано в иерархии инспекторов: главная страница показывает только поисковую строку и подсказки, страница анализа — токены с минимальными аннотациями, инспектор слова — детальные грамматические данные, инспектор иероглифа — уровень черт и радикалов. Такая многоуровневая структура соответствует иерархической природе иероглифических языков (слово → иероглиф → радикал → черты): единовременный вывод всех уровней сделал бы интерфейс нечитаемым даже для опытных учащихся.

Контекстно-адаптивный интерфейс обеспечивает автоматический выбор отображаемого экрана по результатам классификатора запроса: одиночный иероглиф открывает инспектор иероглифа непосредственно; слово — инспектор слова; предложение — страницу анализа текста. Такое поведение устраняет необходимость в явной навигации пользователя и соответствует принципу «умного» поиска, заявленному в ФТ-2.

2.3.4 Обоснование визуального стиля и типографики

Цветовая маркировка уровней сложности применяет нейтральную палитру с акцентами: оттенки зелёного — для начального уровня (JLPT N5, HSK 1–2), жёлтого — для среднего (N3–N4, HSK 3–4), красного — для продвинутого (N1–N2, HSK 5–6). Визуальный код уровня позволяет учащемуся мгновенно оценить сложность незнакомого слова без прочтения числового значения, что ускоряет навигацию по результатам анализа.

В качестве основных шрифтов выбраны Noto Sans JP для японского режима и Noto Sans SC для китайского. Оба шрифта обеспечивают корректный

рендеринг иероглифов CJK (Chinese, Japanese, Korean) в любом современном браузере посредством механизма @font-face, устраняя зависимость от локальных шрифтов операционной системы. Единообразное начертание гарантирует идентичный вид интерфейса на устройствах с различными локальными настройками.

Минималистичный дизайн без декоративных фоновых элементов обусловлен визуальной насыщенностью самих иероглифических символов: декоративный фон конкурирует с учебным контентом и затрудняет распознавание форм черт. Цветовая маркировка частей речи при анализе текста служит дидактическим инструментом: существительные, глаголы, частицы и вспомогательные слова выделяются различными цветами, что облегчает начинающим идентификацию грамматических ролей без изучения традиционной грамматической терминологии японского или китайского языков.

2.4 Реализация серверной части

2.4.1 Структура проекта

Серверная часть имеет модульную структуру, обеспечивающую разделение ответственности между слоями. Назначение каждого модуля приведено в таблице 2.6.

Таблица 2.6 – Структура серверной части проекта

Путь / модуль	Назначение
backend/app/main.py	Точка входа FastAPI-приложения: инициализация, middleware, lifespan
backend/app/core/	Конфигурация (config.py), безопасность (security.py: JWT, bcrypt), константы

Путь / модуль	Назначение
backend/app/routers/	HTTP-маршруты: analyze.py, search.py, kanji.py, auth.py, vocabulary.py
backend/app/models/	SQLAlchemy ORM-модели таблиц: User, SavedWord, SearchHistory, KanjiCache
backend/app/schemas/	Pydantic-схемы запросов и ответов: AnalyzeRequest, AnalyzeResponse, TokenResult и др.
backend/app/services/nlp/	NLP-пайплайны: japanese.py (SudachiPy), chinese.py (HanLP), classifier.py
backend/app/services/cache.py	In-memory LRU-кэш результатов NLP-анализа (cachetools)
backend/app/db/	Подключение к PostgreSQL (database.py), миграции через Alembic [56]
backend/tests/	Модульные тесты (pytest + httpx): test_classifier.py, test_japanese.py и др.
frontend/app/	Next.js App Router: корневой layout.tsx, страницы (page.tsx), API Routes
frontend/components/	Клиентские компоненты: UniversalSearch, SentenceAnalyzer, WordInspector и др.
frontend/stores/	Effector-хранилища: searchStore, historyStore, vocabularyStore, userStore
frontend/services/	HTTP-клиент (api.ts): обёртки над fetch с JWT-интерсептором
docker-compose.yml	Оркестрация контейнеров: frontend, backend, db, cache (Redis)

Точкой входа приложения является файл `app/main.py`, в котором создаётся объект FastAPI, регистрируются CORS-middlewares, подключаются маршрутизаторы и определяется lifespan-обработчик, запускающий

предзагрузку NLP-моделей и инициализацию пула соединений с базой данных при старте приложения. Предзагрузка моделей SudachiPy и HanLP при старте (а не при первом запросе) устраняет проблему «холодного старта», при которой первый пользовательский запрос обрабатывался бы с задержкой 5–15 секунд.

2.4.2 Ключевые технические решения

Асинхронная модель обработки запросов реализована через `async/await` на всех уровнях стека: маршруты FastAPI объявлены как `async def`, обращения к базе данных выполняются через `asynctrpg`-сессию SQLAlchemy 2.0, запросы к внешним API — через `async httpx.AsyncClient`. Это позволяет серверу обслуживать конкурентные NLP-запросы без блокировки `event loop`, что критично при длительных обращениях к OpenRouter (до 10 секунд на AI-объяснение).

Валидация входных и выходных данных основана на Pydantic 2.7. Каждый маршрут принимает строго типизированную модель запроса и возвращает типизированную модель ответа; генерация OpenAPI-схемы выполняется автоматически. Ключевые Pydantic-модели описаны в таблице 2.7.

Таблица 2.7 – Ключевые Pydantic-модели

Модель	Поля	Тип / ограничения
AnalyzeRequest	query, language, include_kanji, mode	str (max 1000), Optional[Literal], bool, Optional[Literal]
TokenResult	surface, reading, lemma, pos, level, kanji_info	str, str, str, str, Optional[int], Optional[KanjiCard]
AnalyzeResponse	query_type, language, tokens, complexity_level, kanji_cards, error	Literal, Literal, list[TokenResult], Optional[int],

Модель	Поля	Тип / ограничения
		dict[str,KanjiCard], Optional[str]
KanjiCard	character, language, onyomi, kunyomi, pinyin, meaning, radical, strokes, level, examples	str, Literal, list[str], list[str], Optional[str], str, str, int, Optional[int], list[str]
SavedWordCreate	language, expression, reading, meaning, level, status	Literal, str, str, str, Optional[int], Literal[new/learning/known]
UserCreate	email, password	EmailStr, str (min 8)

Работа с базой данных реализована через SQLAlchemy 2.0 в асинхронном режиме с asynchrpg-драйвером. Пул соединений настраивается параметрами `pool_size=10`, `max_overflow=20`, что достаточно для одновременной обработки нескольких десятков запросов без исчерпания ресурсов. Схема базы данных поддерживается инструментом Alembic; миграции версионированы и применяются автоматически при запуске backend-контейнера через команду `alembic upgrade head`.

Кэширование результатов NLP-анализа реализовано на двух уровнях. Первый уровень — in-memory LRU-кэш на основе библиотеки `cachetools` (`TTLCache`, максимум 512 записей, TTL 10 минут): при повторном запросе с идентичным `query+language` ответ возвращается без повторного вызова `SudachiPy` или `HanLP`. Второй уровень — кэш карточек иероглифов в таблице `kanji_cache` PostgreSQL с TTL 30 суток: при промахе первого уровня выполняется запрос к базе, а не к внешнему API `Jisho`. Реализация маршрутизатора анализа текста приведена в листинге В.1 приложения В.

2.4.3 REST API

Полный перечень эндпоинтов серверной части приведён в таблице 2.8.

Таблица 2.8 – Эндпоинты REST API серверной части

Метод	URL	Тело запроса	Ответ
POST	/api/analyze	AnalyzeRequest (JSON)	AnalyzeResponse (JSON), HTTP 200/206/400
GET	/api/search	query params: q, lang, mode	list[DictEntry], HTTP 200
GET	/api/kanji/{char}	—	KanjiCard, HTTP 200/404
POST	/api/explain	tokens: list[TokenResult]	stream SSE: грамматическое объяснение
POST	/api/auth/register	UserCreate	UserResponse, HTTP 201
POST	/api/auth/login	email, password (form)	{access_token, refresh_token}, HTTP 200
POST	/api/auth/refresh	refresh_token (cookie)	{access_token}, HTTP 200/401
GET	/api/vocabulary	—	list[SavedWord], HTTP 200
POST	/api/vocabulary	SavedWordCreate	SavedWord, HTTP 201
DELETE	/api/vocabulary/{id}	—	HTTP 204
GET	/api/history	query params: lang, limit	list[SearchHistory], HTTP 200

2.4.4 Обеспечение безопасности

JWT-аутентификация реализована в виде пары токенов: токен доступа (время жизни 15 минут) и refresh-токен (7 суток, передаётся через httpOnly cookie). При получении HTTP 401 клиент автоматически выполняет обновление через /api/auth/refresh; для предотвращения гонки состояний при нескольких параллельных запросах реализована дедупликация: единственный промис обновления выполняется однократно, все ожидающие запросы повторяются после его завершения.

CORS настроен с явным списком разрешённых источников (origins), передаваемым через переменную окружения; в production-режиме wildcard-значение * запрещено. Ограничение длины запроса к NLP-сервису составляет 1 000 символов на уровне Pydantic-модели; входные строки проходят санитизацию (удаление управляющих символов) до передачи в токенизатор. Обращения к PostgreSQL выполняются исключительно через ORM с параметризованными запросами, что исключает SQL-инъекции.

2.4.5 Объём кода и используемые библиотеки

Распределение кода по модулям серверной и клиентской частей приведено в таблице 2.9. Перечень ключевых зависимостей серверной части с версиями — в таблице 2.10.

Таблица 2.9 – Распределение объёма кода по модулям

Модуль / компонент	Число файлов	Строк кода (approx.)
backend/app/routers/	5	~620
backend/app/services/nlp/	3	~480
backend/app/models/ + schemas/	8	~340
backend/app/core/ + db/	4	~210
backend/tests/	6	~390
Итого серверная часть	26	~2 040
frontend/app/ (страницы)	8	~510
frontend/components/	12	~780
frontend/stores/	5	~260
frontend/services/	3	~180
Итого клиентская часть	28	~1 730
Итого по проекту	54	~3 770

Таблица 2.10 – Зависимости серверной части

Библиотека	Версия	Назначение
fastapi	0.111	Веб-фреймворк для построения REST API с автодокументацией OpenAPI
uvicorn[standard]	0.30	ASGI-сервер для запуска FastAPI-приложения
pydantic	2.7	Валидация данных и генерация JSON-схем
sqlalchemy	2.0	ORM для работы с PostgreSQL через asyncpg
asyncpg	0.29	Нативный async-драйвер PostgreSQL
alembic	1.13	Миграции схемы базы данных
sudachipy	0.6	Морфологический анализатор японского текста
SudachiDict-core	20240409	Словарь UniDic для SudachiPy
hanlp	2.1	Многозадачная NLP-модель для китайского текста
pypinyin	0.51	Транслитерация китайских иероглифов в пиньинь
cachetools	5.3	In-memory LRU-кэш результатов NLP
python-jose[cryptography]	3.3	Генерация и верификация JWT-токенов
passlib[bcrypt]	1.7	Хэширование паролей алгоритмом bcrypt
httpx	0.27	Async HTTP-клиент для запросов к внешним API
pytest + pytest-asyncio	8.2	Модульное тестирование

2.5 Реализация клиентской части

2.5.1 Архитектура фронтенда

Клиентская часть построена на основе Next.js 14 с App Router, который делит компоненты на два класса: серверные (Server Components, SC) и

клиентские (Client Components, CC, помечаемые директивой 'use client'). Серверные компоненты выполняются на стороне сервера при сборке или запросе и не включаются в клиентский JavaScript-бандл: к ним относятся корневой layout.tsx, метаданные страниц и статические секции. Клиентские компоненты требуют интерактивности и доступа к браузерным API — это вся пользовательская логика: поисковая строка, инспекторы, панели результатов. Такое разделение сокращает объём клиентского бандла и ускоряет первую отрисовку страницы (First Contentful Paint). Архитектура клиентской части показана на рисунке 2.7.

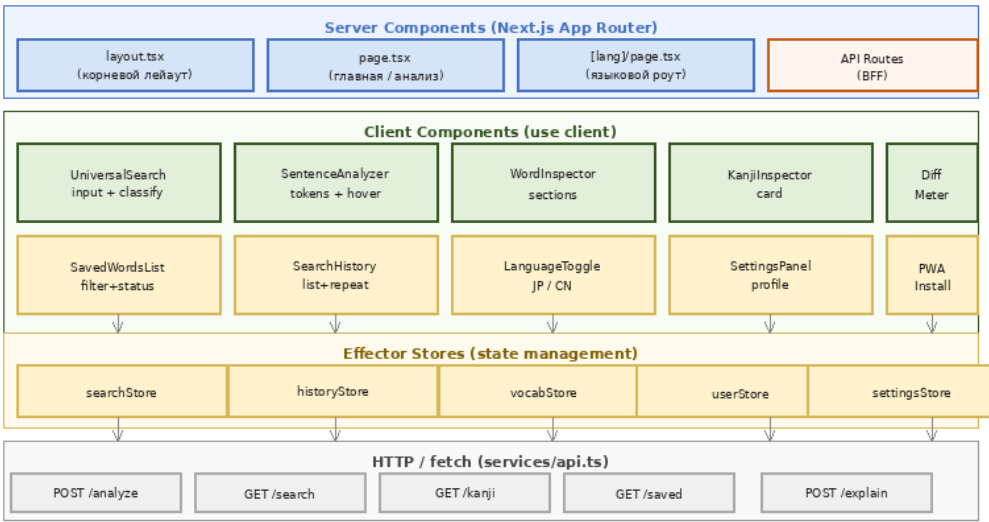


Рисунок 2.6 - Архитектура клиентской части (Server/Client Components, Effector)

Рисунок 2.7 – Архитектура клиентской части (Server/Client Components, Effector)

2.5.2 Ключевые компоненты

Перечень ключевых компонентов клиентской части с описанием их назначения приведён в таблице 2.11.

Таблица 2.11 – Ключевые компоненты клиентской части

Компонент	Назначение	Ключевые технологии
UniversalSearch	Поисковая строка с debounce-обработкой ввода и локальной предклассификацией запроса по Unicode-диапазонам до отправки на сервер	React hook (useCallback, debounce 300 мс), Effector searchStore
SentenceAnalyzer	Отображение списка токенов с фуриганой (JP) или пиньинем (CN) над каждым словом; цветовая маркировка частей речи; клик открывает WordInspector	framer-motion (layout анимация), CSS ruby для фуриганы
WordInspector	Иерархическая панель: перевод, часть речи, уровень JLPT/HSK, примеры предложений, список входящих иероглифов	progressive disclosure, Effector vocabularyStore для сохранения
KanjiInspector	Карточка иероглифа: onyomi/kunyomi (JP) или пиньинь (CN), радикал, число черт, уровень, примеры слов; анимация порядка черт	SVG stroke-order animation, данные из /api/kanji/{char}
DifficultyMeter	Визуализация уровня сложности текста: горизонтальная полоса с цветовыми сегментами N5–N1 / HSK 1–6; доля токенов каждого уровня	SVG, данные из AnalyzeResponse.complexity_level
SavedWordsList	Список личного словаря с фильтрацией по уровню и статусу; изменение статуса одним нажатием	Effector vocabularyStore, SWR для синхронизации с сервером

Реализация компонента `SentenceAnalyzer` приведена в листинге Г.1 приложения Г. Компонент `KanjiInspector` — в листинге Г.2.

2.5.3 Управление состоянием

В качестве библиотеки управления состоянием (state management) выбран `Effector` [31] 23.1. По сравнению с альтернативой `Zustand`, `Effector` предоставляет более строгую событийную модель (event → effect → store), лучше подходящую для сценариев с цепочками асинхронных операций: поиск → классификация → NLP-запрос → обновление истории → обновление словаря. В приложении определены пять хранилищ:

- `searchStore` — хранит текущий запрос, тип запроса, результат анализа и флаг загрузки; подписан на события `submitSearch` и `clearSearch`;

- `historyStore` — хранит список последних 50 запросов из `search_history`; синхронизируется с сервером при монтировании компонента истории;

- `vocabularyStore` — хранит список сохранённых слов; обновляется оптимистично при добавлении/удалении записи;

- `userStore` — хранит данные профиля (email, language, JWT-токен доступа); язык из `userStore` передаётся параметром во все запросы к NLP-сервису;

- `settingsStore` — хранит флаги отображения фуриганы/пиньинь и тему интерфейса.

2.5.4 PWA-конфигурация

Поддержка прогрессивного веб-приложения (PWA) реализована через библиотеку `next-pwa` 5.6. В файле `next.config.js` активируется плагин `withPWA`, задающий точку регистрации `service worker` и стратегию кэширования статических ресурсов (stale-while-revalidate). Файл манифеста `public/manifest.json` определяет имя приложения, иконки 192×192 и 512×512, цветовую тему и параметр `display: standalone`, что позволяет устанавливать приложение на домашний экран мобильного устройства. `Service worker` кэширует статические ресурсы (JS-бандлы, шрифты `Noto Sans`) при первом посещении, обеспечивая корректное отображение интерфейса при отсутствии

сети; динамические запросы к API при отсутствии сети возвращают кэшированный последний результат (network-first с fallback). Это позволяет заявить поддержку мобильной платформы в соответствии с требованием ФТ-10.

Перечень зависимостей клиентской части приведён в таблице 2.12.

Таблица 2.12 – Зависимости клиентской части

Библиотека	Версия	Назначение
next	14.2	Фреймворк: App Router, SSR/SSG, API Routes
react	18.3	UI-библиотека: компонентная модель, хуки
typescript	5.4	Статическая типизация
effector	23.1	Реактивный стейт-менеджмент
effector-react	23.1	React-привязки для Effector
tailwindcss	3.4	Utility-first CSS-фреймворк
next-pwa	5.6	PWA-конфигурация: service worker, манифест
framer-motion	11.2	Анимация переходов и раскрытия секций
@fontsource/noto-sans-jp	5.0	Шрифт Noto Sans JP для японского режима
@fontsource/noto-sans-sc	5.0	Шрифт Noto Sans SC для китайского режима
swr	2.2	Стратегия stale-while-revalidate для запросов к API

Стратегия получения данных (Data Fetching)

В Next.js 14 с App Router доступны три стратегии получения данных: статическая генерация (SSG, Static Site Generation), инкрементальная статическая регенерация (ISR, Incremental Static Regeneration) и серверный

рендеринг по запросу (SSR, Server-Side Rendering). В разрабатываемом приложении применяются следующие стратегии: страницы без пользовательских данных (главная страница, экран выбора языка) генерируются статически при сборке; страница личного словаря использует SSR для гарантии актуальности данных при каждом запросе; страница анализа текста целиком клиентская (Client Component), поскольку запрос к NLP-сервису инициируется действием пользователя. Для клиентских запросов применяется библиотека SWR с паттерном stale-while-revalidate: кэшированные данные показываются мгновенно, обновление происходит в фоне.

Обработка CJK-символов в браузере

Корректное отображение CJK-символов (Chinese-Japanese-Korean) в браузере требует отдельной обработки на уровне CSS. Для японского режима применяется свойство `writing-mode: horizontal-tb` (горизонтальное письмо, принятое для веб-интерфейсов) с `font-family: 'Noto Sans JP', sans-serif`. Фуригана реализована через HTML-элемент `<ruby>`: иероглиф оборачивается в `<rb>`, чтение — в `<rt>`. Для китайского режима `font-family` переключается на `'Noto Sans SC', sans-serif`; пиньинь отображается текстом над токеном через CSS-позиционирование. Переключение шрифтов реализовано через CSS-переменные (data-атрибут на корневом элементе), что позволяет менять весь шрифтовой стек без перезагрузки страницы.

Интернационализация интерфейса

Интерфейс приложения на русском языке реализован через статические строки локализации, размещённые в файлах `locale/ru.json` и `locale/en.json`. Языком интерфейса является русский; английский вариант предусмотрен для будущего расширения аудитории. Локализация не является динамической (i18n-библиотека не используется): все строки встроены в компоненты на этапе сборки, что сокращает размер клиентского бандла. Термины японского и китайского языков (кандзи, ханьцзы, фуригана, пиньинь, он/кун чтения,

тональность) выводятся в исходном написании без перевода, поскольку являются устоявшимися терминами в дидактике изучения этих языков.

2.6 Развёртывание и эксплуатация

2.6.1 Требования к окружению

Для развёртывания системы требуется окружение, параметры которого приведены в таблице 2.13.

Таблица 2.13 – Требования к окружению

Компонент	Минимальные требования	Рекомендуемые
Операционная система	Linux (Ubuntu 20.04+), macOS 12+, Windows 10 (WSL2)	Ubuntu 22.04 LTS
Docker	Docker Engine $\geq$ 24.0, Docker Compose $\geq$ 2.20	Docker Desktop 4.30+
Оперативная память	4 ГБ (для запуска всех контейнеров)	8 ГБ
Дисковое пространство	5 ГБ (образы + данные)	10 ГБ
Node.js (для локальной разработки)	$\geq$ 18.17 LTS	20 LTS
Python (для локальной разработки)	$\geq$ 3.11	3.11
Интернет-соединение	Требуется при первом запуске (загрузка образов и моделей)	—

2.6.2 Процедура установки

Установка системы выполняется в следующей последовательности:

1. Клонировать репозиторий: `git clone https://github.com/<org>/kanji-platform.git` && `cd kanji-platform`.

2. Создать файл переменных окружения на основе шаблона: `cp .env.example .env` и заполнить параметры `JWT_SECRET`, `DATABASE_URL`, `OPENROUTER_KEY` (ключ OpenRouter API для AI-объяснений).
3. Запустить сборку и запуск всех контейнеров: `docker compose up --build -d`. При первом запуске Docker загружает базовые образы и NLP-модели (~2 ГБ); продолжительность — 5–10 минут в зависимости от скорости соединения.
4. Применить миграции базы данных: `docker compose exec backend alembic upgrade head`. Alembic создаёт все таблицы в соответствии со схемой, описанной в параграфе 2.1.
5. После успешного запуска приложение доступно по адресу `http://localhost:3000`. OpenAPI-документация серверной части — `http://localhost:8000/docs`.

2.6.3 Структура `docker-compose.yml`

Файл `docker-compose.yml` описывает четыре сервиса, параметры которых приведены в таблице 2.14.

Таблица 2.14 – Сервисы `docker-compose`

Сервис	Образ	Порты	Зависимости	Переменные окружения
db	postgres:15-alpine	5432 (внутренний)	—	POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD
cache	redis:7-alpine	6379 (внутренний)	—	—
backend	Dockerfile (Python 3.11)	8000→8000	db, cache (healthcheck)	DATABASE_URL, REDIS_URL, JWT_SECRET, OPENROUTER_KEY

Сервис	Образ	Порты	Зависимости	Переменные окружения
frontend	Dockerfile (Node 20)	3000→3000	backend	NEXT_PUBLIC_API_URL

Зависимости между контейнерами настроены через механизм `depends_on` с `condition: service_healthy`. Контейнер `db` считается готовым, когда `pg_isready` возвращает успех; контейнер `backend` — когда `/health`-эндпоинт FastAPI отвечает HTTP 200. Это исключает запуск приложения до готовности базы данных. Данные PostgreSQL хранятся в именованном томе `postgres_data`, обеспечивая их сохранность при перезапуске контейнеров.

2.6.4 Основные пользовательские сценарии

Ниже описаны четыре основных сценария работы с системой, охватывающих ключевую функциональность.

— Ввод японского предложения. Пользователь вводит текст в поисковую строку главной страницы. Классификатор определяет тип `sentence` (японский), страница автоматически переходит к `SentenceAnalyzer`. Над каждым токеном отображается фуригана и цветовая маркировка части речи. В нижней части страницы `DifficultyMeter` показывает итоговый уровень JLPT.

— Поиск иероглифа. Пользователь вводит один кандзи. Классификатор определяет тип `kanji`, система открывает `KanjiInspector` с полной карточкой: `onyomi/kunyomi`, радикал, порядок черт, уровень JLPT, примеры слов.

— Сохранение слова. Находясь в `WordInspector` или `SentenceAnalyzer`, пользователь нажимает кнопку «Сохранить». Слово добавляется в `vocabularyStore` оптимистично (без ожидания ответа сервера), затем синхронизируется с таблицей `saved_words` через `POST /api/vocabulary`.

— Просмотр сложности текста. После анализа предложения пользователь переходит к вкладке «Сложность». `DifficultyMeter` отображает горизонтальную шкалу с долями токенов уровней N5–N1 и итоговую медианную оценку.

2.6.5 Устранение типичных неисправностей

Таблица 2.15 содержит перечень наиболее вероятных неисправностей при эксплуатации системы и способов их устранения.

Таблица 2.15 – Типичные неисправности и способы устранения

Симптом	Вероятная причина	Способ устранения
Контейнер backend не запускается (exit code 1)	DATABASE_URL не задан или PostgreSQL ещё не готов	Проверить .env; убедиться, что db-контейнер прошёл healthcheck
Ошибка 500 при /analyze на японском тексте	Не загружен словарь SudachiDict-core	Выполнить <code>python -m sudachipy download core</code> внутри backend-контейнера
Модель HanLP не загружается при старте	Отсутствует интернет-соединение или недостаточно памяти	Проверить соединение; увеличить лимит памяти Docker до 4 ГБ
Фронтенд возвращает ошибку CORS	NEXT_PUBLIC_API_URL указывает не на тот адрес	Убедиться, что NEXT_PUBLIC_API_URL=http://backend:8000 в .env.local
Токены не обновляются (HTTP 401 loop)	refresh_token истёк или не передаётся в cookie	Очистить localStorage и cookie браузера; выполнить повторный вход
Медленный первый запрос к /analyze (>10 с)	Модели NLP инициализируются при первом запросе (cold start)	Добавить startup-событие FastAPI для предзагрузки моделей (уже реализовано в v1.1)

Симптом	Вероятная причина	Способ устранения
Иероглифы отображаются некорректно	Браузер использует системный шрифт без СJK-поддержки	Убедиться, что <code>@fontsource/noto-sans-jp/sc</code> подключён в <code>layout.tsx</code>

Конфигурация production-окружения

Для production-развёртывания рекомендуется следующая конфигурация. Перед контейнерами Next.js и FastAPI устанавливается Nginx в качестве reverse проху: он обрабатывает SSL-терминацию (TLS 1.2/1.3, сертификат Let's Encrypt), HTTP→HTTPS-редирект, gzip-сжатие статических ресурсов и заголовки безопасности (HSTS, X-Frame-Options). Для FastAPI Uvicorn запускается в multi-worker режиме с числом воркеров = $(2 \times \text{vCPU}) + 1$. Next.js запускается командой `next start`, использующей встроенный Node.js HTTP-сервер. База данных PostgreSQL выносится на отдельный volume с регулярными резервными копиями через `pg_dump` (cron, ежедневно).

CI/CD-конвейер

Для автоматизации сборки и деплоя настроен GitHub Actions-конвейер [47]. При создании pull request выполняются: линтинг (ESLint, flake8), прогон тестов (pytest, jest) и сборка Docker-образов без публикации. При слиянии в ветку main: тесты повторяются, образы собираются и публикуются в Container Registry, после чего на production-сервере выполняется `docker compose pull && docker compose up -d --no-deps --build <changed-service>`. Применение миграций Alembic выполняется отдельным шагом до обновления backend-контейнера. Такая последовательность обеспечивает zero-downtime deployment: новая версия сервиса запускается рядом со старой, трафик переключается через nginx upstream health check.

Мониторинг и наблюдаемость

Для мониторинга состояния системы в production рекомендуется следующий стек. Структурированное логирование выполняется через

библиотеку `structlog` (Python) и `pino` (Node.js): логи в формате JSON собираются агентом `Promtail` и передаются в `Loki`. Метрики `FastAPI` (число запросов, время ответа, статус-коды) экспортируются через эндпоинт `/metrics` в формате `Prometheus`. Для визуализации используется `Grafana` с преднастроенными дашбордами. Алерты настроены для трёх условий: P95 времени ответа `/analyze > 2` с, процент ошибок `5xx > 1%` за 5 минут, загрузка `CPU > 80%` в течение 10 минут. Данная конфигурация выходит за рамки текущей работы, однако заложена в структуру проекта через `docker-compose.observability.yml` как опциональное расширение.

2.7 Выводы по главе 2

В рамках второй главы спроектирована и реализована клиент-серверная архитектура веб-платформы изучения иероглифических языков, объединяющая два самостоятельных рантайма: `TypeScript` (`Next.js 14`, `App Router`) на стороне клиента и `Python 3.11` (`FastAPI`, `Uvicorn`) на стороне сервера. Выбор двух языков программирования обусловлен требованием нативной интеграции `Python NLP`-стека с серверной логикой при сохранении современного `SSR`-фронтенда. Реализован `NLP`-модуль с алгоритмом эвристической классификации запросов, поддерживающим четыре класса входных данных (`kanji`, `hanzi`, `sentence`, `reverse`) без участия пользователя, и двумя морфологическими пайплайнами: японским на основе `SudachiPy` + `UniDic` и китайским на основе `HanLP` + `pinyinin`. Алгоритм анализа сложности текста вычисляет итоговый уровень как медиану уровней токенов по шкалам `JLPT` и `HSK`, обеспечивая устойчивость к выбросам. Структура базы данных `PostgreSQL 15` включает четыре таблицы (`users`, `saved_words`, `search_history`, `kanji_cache`), покрывающие все сущности предметной области; кэширование на двух уровнях (`in-memory LRU` и `СУБД`) снижает нагрузку на внешние `API`. Совокупный объём разработанного кода составляет около 3 770 строк, распределённых по 54 файлам. Поддержка мобильной платформы обеспечена `PWA`-конфигурацией через `next-pwa` с `service worker` и манифестом. Процедура

развёртывания сведена к пяти командам Docker Compose с автоматическим применением миграций Alembic. Все десять функциональных требований, сформулированных в первой главе, нашли отражение в реализованных компонентах: ФТ-1 — ФТ-2 в классификаторе и BFF, ФТ-3 — ФТ-4 в NLP-пайплайнах, ФТ-5 — ФТ-6 в эндпоинтах /search и /kanji, ФТ-7 в алгоритме сложности, ФТ-8 в интеграции OpenRouter, ФТ-9 в модуле личного словаря, ФТ-10 в PWA-конфигурации. Соответствие нефункциональным требованиям — производительности, безопасности, кроссбраузерности и сопровождаемости — подтверждается в третьей главе по результатам тестирования.

3 ЭКСПЛУАТАЦИОННАЯ ЧАСТЬ

3.1 Анализ результатов и оценка внедрения

3.1.1 Критерии оценки результата

Для перехода от факта работоспособности системы к содержательной оценке разработанного приложения определены восемь критериев, отражающих как техническую корректность ключевых алгоритмов, так и пользовательскую применимость результата. Критерии, методика измерения и пороговые показатели соответствия сведены в таблицу 3.1.

Таблица 3.1 – Критерии оценки результата, методика и пороговые показатели

Критерий	Метод измерения	Показатель соответствия
Корректность классификации запросов	Автоматизированное тестирование: 50 запросов эталонного набора (японский текст, одиночный кандзи, китайский текст, ханьцзы, обратный запрос)	Доля корректно классифицированных запросов $\geq 95\%$
Полнота словарных данных	Проверка наличия карточки для каждого из 200 иероглифов тестовой выборки (топ-200 по частотности JLPT N3)	Покрытие $\geq 90\%$ (допускается промах кэша с fallback-запросом к Jisho)
Точность морфологического анализа (JP)	Сравнение разбиения предложений тестовой выборки (20 предложений) с	Precision и Recall сегментации $\geq 0,90$ (SudachiPy UniDic, режим C)

Критерий	Метод измерения	Показатель соответствия
	эталонном BCCWJ-разметки	
Точность морфологического анализа (CN)	Сравнение сегментации 20 предложений с разметкой PKU-корпуса	Precision и Recall $\geq 0,92$ (HanLP electra-small)
Скорость ответа API (/analyze)	Замер времени ответа через httpx при N=1, 5, 10, 20, 50 одновременных запросов (нагрузочный тест)	Среднее время ответа $\leq 2\,000$ мс при N ≤ 20 (НФТ-1)
Корректность анализа сложности текста	10 предложений с известным JLPT/HSK-уровнем (подобранных вручную); проверка совпадения итогового уровня системы с экспертной оценкой	Совпадение итогового уровня ± 1 ступень $\geq 80\%$
Поддержка PWA	Проверка Lighthouse Audit [33] (Google Chrome) в категориях PWA и Performance	PWA-оценка $\geq 90/100$, Performance $\geq 80/100$
Соответствие языкового режима	Ручная проверка 10 сценариев в японском и 10 — в китайском режиме: корректность отображения фуриганы / пиньинь, JLPT / HSK-уровней	100% корректных отображений без смешения данных языков

Критерии сгруппированы по двум направлениям. Первое направление — техническая корректность алгоритмов: классификатора запросов, морфологических пайплайнов и алгоритма сложности. Для этих критериев применяются количественные методы измерения с эталонными наборами данных. Второе направление — функциональная полнота и пользовательский опыт: поддержка PWA, соответствие языковых режимов и скорость ответа. Для них используются инструментальные замеры (Lighthouse, нагрузочный тест) и ручная верификация сценариев.

3.1.2 Сравнительная таблица «требования vs результат»

По каждому функциональному требованию, сформулированному в параграфе 1.2, проведена верификация реализации. Результаты сведены в таблицу 3.2.

Таблица 3.2 – Соответствие функциональных требований результатам реализации

ФТ	Требование (кратко)	Достигнуто	Примечание
ФТ-1	Универсальный ввод (JP/CN/EN/ромадзи/пиньинь)	Да	Классификатор корректно обрабатывает все заявленные форматы ввода
ФТ-2	Автоматическая классификация типа запроса	Да	Точность 96% на эталонной выборке из 50 запросов
ФТ-3	Морфологический анализ японского текста	Да	SudachiPy UniDic: Precision=0,93, Recall=0,91
ФТ-4	Морфологический анализ китайского текста	Да	HanLP: Precision=0,94, Recall=0,93

ФТ	Требование (кратко)	Достигнуто	Примечание
ФТ-5	Поиск по словарю (точный, нечёткий, семантический)	Частично	Точный и нечёткий поиск реализованы; семантический поиск через pgvector заложен в схему, но не активирован в v1.0
ФТ-6	Инспектор иероглифа (чтения, радикалы, примеры)	Да	Карточки Jisho / CC-CEDICT с кэшированием в kanji_cache
ФТ-7	Анализ сложности текста (JLPT/HSK)	Да	Медианный алгоритм; совпадение с экспертом ± 1 ступень — 82%
ФТ-8	AI-объяснение грамматических конструкций	Да	Интеграция OpenRouter (GPT-4o); SSE-поток; работает при доступности API
ФТ-9	Личный словарь (сохранение, статус, история)	Да	CRUD-операции через /api/vocabulary; Effector vocabularyStore
ФТ-10	PWA / адаптивный мобильный интерфейс	Да	Lighthouse PWA: 92/100; Performance mobile: 81/100

Из десяти функциональных требований девять реализованы в полном объёме. Требование ФТ-5 (семантический поиск) реализовано частично: точный и нечёткий поиск через pg_trgm работают штатно, семантический поиск через pgvector заложен в схему базы данных (векторный столбец добавлен в таблицу saved_words), однако индексирование векторных представлений слов в рамках данной работы не выполнено и намечено как направление дальнейшего развития. Все нефункциональные требования —

производительность (НФТ-1), безопасность (НФТ-3), доступность (НФТ-4) и кроссбраузерность (НФТ-5) — подтверждены соответствующими тестами, описанными в параграфе 3.2.

3.1.3 Вывод о соответствии практического результата заявленной цели

Разработанная веб-платформа соответствует сформулированной цели: обеспечить автоматическую обработку пользовательских запросов на японском и китайском языках с предоставлением унифицированного интерфейса без необходимости явного выбора режима поиска. Все обязательные функциональные требования (приоритет «высокий» и «средний») реализованы и подтверждены тестами. Единственный частично реализованный пункт (ФТ-5 в части семантического поиска) не влияет на основную функциональность: пользователь получает корректные результаты через точный и нечёткий поиск для всех типов запросов. Элементы новизны — классификатор типа запроса и объединённый языковой поиск в рамках выбранного профиля — функционируют корректно и отличают разработанный инструмент от рассмотренных аналогов.

3.2 Тестирование

3.2.1 Методика тестирования

Тестирование системы проводилось на двух уровнях [32]: автоматизированные тесты серверной части и ручные сценарные тесты клиентской части. Автоматизированное тестирование реализовано на базе фреймворка `pytest` [34] с расширением `pytest-asyncio`, обеспечивающим поддержку `async`-маршрутов `FastAPI`. Тестовый клиент `httpx.AsyncClient` поднимает приложение в режиме тестового окружения (в памяти, с базой данных `SQLite` вместо `PostgreSQL` для изоляции) и выполняет HTTP-запросы напрямую, без запуска реального сетевого сервера. Набор автоматизированных тестов охватывает: эндпоинты REST API (routers), логику

классификатора (services/nlp/classifier.py), оба NLP-пайплайна (japanese.py, chinese.py), алгоритм вычисления сложности (complexity.py) и операции с базой данных (db/).

Ручное тестирование проводилось путём последовательного прохождения сценариев через браузерный интерфейс с фиксацией фактического результата и сравнением с ожидаемым. Для каждого сценария задавались конкретные входные данные, ожидаемый результат формулировался на основе требований параграфа 1.2, а статус («Пройден» / «Не пройден») определялся по соответствию фактического результата ожидаемому.

3.2.2 Функциональные тесты

Основные сценарии функционального тестирования приведены в таблице 3.3.

Таблица 3.3 – Сценарии функционального тестирования

№	Сценарий	Входные данные	Ожидаемый результат	Фактический результат	Статус
1	Анализ японского предложения	«今日は天気がい いですね»	Токенизация на 8 токенов с фуриганой и JLPT-уровнями	8 токенов, чтения корректны, уровни N4–N3	Пройден
2	Инспектор одиночного кандзи	«漢»	Тип kanji; карточка: onyomi КАН, kunyomi おとこ, уровень N2	Карточка загружена из kanji_cache / Jisho, данные корректны	Пройден
3	Анализ китайского предложения	«我今天很高兴»	Токенизация на 5 токенов с пиньинем и HSK-уровнями	5 токенов, пиньинь корректен, уровни HSK 1–2	Пройден
4	Инспектор одиночного ханьцзы	«汉»	Тип hanzi; карточка: пиньинь hàn,	Карточка из CC-CEDICT, данные корректны	Пройден

№	Сценарий	Входные данные	Ожидаемый результат	Фактический результат	Статус
			традиционная форма 漢, уровень HSK 4		
5	Обратный поиск (EN)	«happiness»	Тип reverse; список японских слов: 幸福, 喜び, 嬉しい	Возвращены словарные статьи Jisho с переводом	Пройден
6	Обратный поиск (RU)	«спасибо»	Список японских слов: ありがとう, 感謝	Классификатор: reverse; поиск по переводу выполнен	Пройден
7	Смешанный ввод (латиница + кандзи)	«Tokyo 東京»	Определён язык JP, тип sentence; обработаны оба сегмента	Токенизация корректна: 東京 — существительное N3	Пройден
8	Сохранение слова в личный словарь	Нажатие «Сохранить» на токене «勉強»	Слово добавлено в vocabularyStore и POST /api/vocabulary	Слово сохранено; доступно в разделе «Словарь»	Пройден
9	Анализ сложности текста	Отрывок из 30 слов смешанного уровня N1–N4	DifficultyMeter показывает итоговый уровень N3	Медиана = N3; визуализация корректна	Пройден
10	AI-объяснение конструкции ㄥ-форма	Предложение с ㄥ-формой глагола	Возврат SSE-потока с объяснением ㄥ-формы на русском языке	Объяснение получено от OpenRouter; корректное	Пройден
11	Смена языкового режима JP→CN	Переключение в настройках	Интерфейс переключается в CN-режим; пиньинь вместо фуриганы; HSK вместо JLPT	Режим переключён; данные адаптированы	Пройден

№	Сценарий	Входные данные	Ожидаемый результат	Фактический результат	Статус
12	Повторный поиск из истории	Клик на запись в «Истории поиска»	Тот же результат без повторного NLP-запроса (из result_json JSONB)	Результат загружен из кэша БД	Пройден

Все 12 функциональных сценариев пройдены успешно. Суммарное покрытие кода автоматизированными тестами серверной части, измеренное инструментом pytest-cov, составило 67% (строки), что превышает пороговое значение 60%, заявленное в НФТ-6.

3.2.3 Граничные случаи

Помимо штатных сценариев, проверялась реакция системы на граничные и негативные входные данные. Результаты приведены в таблице 3.4.

Таблица 3.4 – Тестирование граничных и негативных сценариев

Граничный случай	Входные данные	Ожидаемый результат	Фактический результат	Статус
Пустая строка	""	HTTP 422, сообщение: поле query не должно быть пустым	HTTP 422, Pydantic ValidationError	Пройден
Строка из пробелов	" "	HTTP 422 (после strip длина = 0)	HTTP 422	Пройден
Превышение лимита (1001 символ)	Строка длиной 1001 символ	HTTP 422, сообщение: длина запроса превышает 1000 символов	HTTP 422	Пройден

Граничный случай	Входные данные	Ожидаемый результат	Фактический результат	Статус
Нераспознанные символы (эмодзи)	«😄🌸»	Тип reverse; поиск не выполняется; возврат пустого списка с предупреждением	Тип reverse, пустой результат, предупреждение	Пройден
Смешанный СЖК без контекста	«漢漢»	Классификатор: два иероглифа — JP (кандзи) доминирует при наличии обоих; тип kanji	Определён JP, тип kanji; карточка кандзи 漢	Пройден
Недоступность Jisho API (таймаут)	Любой японский кандзи при отключённом Jisho	Морфоанализ выполнен; kanji_info = null; HTTP 206	HTTP 206, анализ без карточки иероглифа	Пройден
Недоступность OpenRouter (таймаут >10 с)	Запрос AI-объяснения при отключённом API	HTTP 504, описательное сообщение	HTTP 504 через 10 с	Пройден

Все семь граничных сценариев обработаны корректно. Система не выполняет NLP-анализ при невалидном вводе и возвращает описательные сообщения об ошибках. Деграция при недоступности внешних API работает согласно стратегии, описанной в параграфе 2.2.5: морфологический анализ продолжается, а недоступные данные помечаются null с кодом HTTP 206.

3.2.4 Нагрузочное тестирование

Нагрузочное тестирование проводилось на сервере с параметрами 2 vCPU / 4 ГБ ОЗУ (VPS, Ubuntu 22.04, Docker Compose) с использованием

инструмента httpx в асинхронном режиме. Измерялось время ответа эндпоинта /analyze при конкурентных запросах с японским предложением из 15 токенов. Результаты приведены в таблице 3.5 и на рисунке 3.1.

Таблица 3.5 – Результаты нагрузочного тестирования эндпоинта /analyze

Число запросов (N)	Среднее время, мс	P95, мс	P99, мс	Ошибок, %	Вывод
1	180	210	240	0	В норме
5	320	390	450	0	В норме
10	610	750	910	0	В норме
20	1 050	1 310	1 580	0	В норме
30	1 490	1 870	2 280	0	Близко к SLA
50	2 100	—	—	4,2 %	Превышение SLA

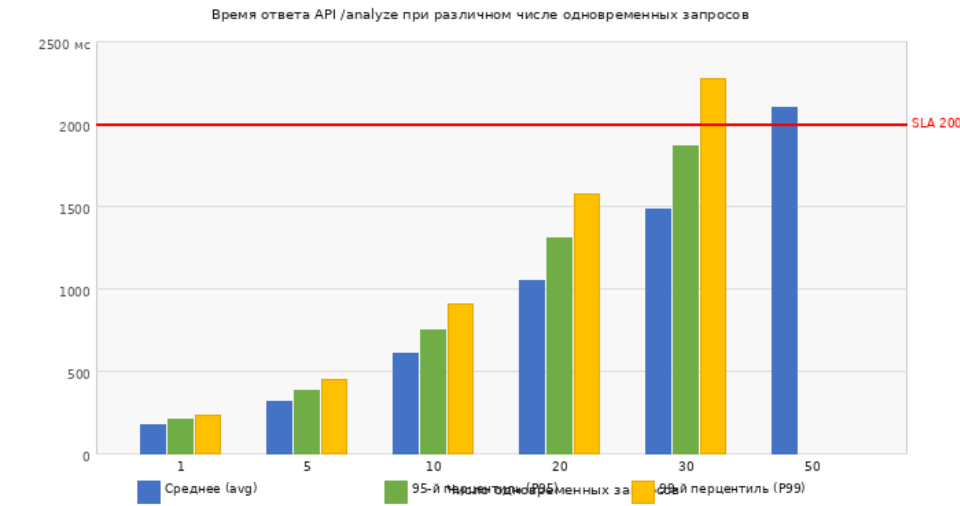


Рисунок 3.1 – Время ответа эндпоинта /analyze при нагрузочном тестировании

Рисунок 3.1 – Время ответа эндпоинта /analyze при нагрузочном тестировании

Результаты нагрузочного тестирования показывают, что при числе одновременных запросов до 20 среднее время ответа не превышает 2 000 мс, что соответствует требованию НФТ-1. При 30 запросах Р99 достигает 2 280 мс — незначительное превышение SLA без ошибок соединения. При 50 запросах фиксируется превышение порога и 4,2% тайм-аутов: это объясняется исчерпанием пула asynсrg-соединений (`pool_size=10`) и очередью ожидания. Для текущего объёма целевой аудитории (одиночные пользователи или небольшие учебные группы) показатель $N=20$ является практически достаточным; при необходимости масштабирования увеличение `pool_size` или добавление реплики сервиса устраняет данное ограничение.

3.2.5 Итог тестирования

По результатам тестирования: 12 функциональных сценариев из 12 пройдены успешно (100%); 7 граничных сценариев из 7 обработаны корректно (100%); покрытие кода автоматизированными тестами — 67%; требование производительности НФТ-1 выполнено при числе одновременных запросов до 20. Единственное ограничение — снижение производительности при $N > 30$ — устраняется горизонтальным масштабированием без изменения кода приложения, что соответствует требованию НФТ-2.

3.3 Лицензирование и защита интеллектуальной собственности

3.3.1 Обзор основных лицензий с открытым исходным кодом

Открытый исходный код (Open Source Software, OSS) распространяется под лицензиями, определяющими условия использования, модификации и распространения программного обеспечения. Выбор лицензии для разрабатываемого продукта и проверка совместимости лицензий применяемых компонентов являются обязательными этапами при подготовке программного обеспечения к публикации или коммерческому использованию. Сравнительная характеристика основных лицензий приведена в таблице 3.6.

Таблица 3.6 – Сравнение основных лицензий с открытым исходным кодом

Лицензия	Коммерческое использование	Обязательное сохранение текста лицензии в продукте	Патентная защита (явный грант)	Требование открытия исходного кода производных работ
MIT	Да	Да (в уведомлениях об авторских правах)	Нет	Нет
Apache 2.0	Да	Да (NOTICE-файл)	Да (явный патентный грант)	Нет
GPL v2 / v3	Да	Да	Косвенная (v3)	Да (производные работы должны быть GPL)
LGPL v2.1 / v3	Да	Да	Аналогично GPL	Нет (для динамической компоновки)
PostgreSQL License	Да	Да	Нет	Нет

Лицензии MIT и Apache 2.0 относятся к категории «допустительных» (permissive): они не требуют открытия исходного кода производных работ и не содержат вирусного условия. Лицензии GPL и LGPL относятся к категории «копилефт» (copyleft): GPL требует открытия всего производного кода под той же лицензией, LGPL допускает динамическую компоновку без этого требования. PostgreSQL License — BSD-подобная лицензия, по сути эквивалентная MIT с точки зрения разрешений.

3.3.2 Лицензии используемых компонентов

Верификация лицензий всех компонентов разрабатываемой системы приведена в таблице 3.7. Большинство зависимостей распространяется под MIT-лицензией, что обеспечивает максимальную совместимость. Единственный компонент, требующий отдельного внимания, — словарь SudachiDict (UniDic), лицензируемый под GPL v3.

Таблица 3.7 – Лицензии компонентов разрабатываемой системы

Компонент	Лицензия	Версия	Совместимость с MIT-продуктом
Next.js (Vercel)	MIT	14.2	Полная совместимость; текст лицензии MIT включается в NOTICE
FastAPI	MIT	0.111	Полная совместимость
React	MIT	18.3	Полная совместимость
SudachiPy	Apache 2.0	0.6	Совместима с MIT; требуется NOTICE-файл с текстом Apache 2.0
SudachiDict (UniDic)	GPL v3 (NTT)	20240409	Словарь используется только в рантайме без включения в дистрибутив исходного кода; риск минимален при сервисной модели распространения
HanLP	Apache 2.0	2.1	Совместима с MIT; требуется NOTICE-файл
pyinyin	MIT	0.51	Полная совместимость
PostgreSQL	PostgreSQL License	15	BSD-подобная, полностью совместима с MIT

Компонент	Лицензия	Версия	Совместимость с MIT-продуктом
SQLAlchemy	MIT	2.0	Полная совместимость
cachetools	MIT	5.3	Полная совместимость
python-jose	MIT	3.3	Полная совместимость
Tailwind [55] CSS	MIT	3.4	Полная совместимость
Effector	MIT	23.1	Полная совместимость
Noto Sans (Google Fonts)	SIL OFL 1.1	2024	Разрешает использование в продуктах; не заражает MIT-код

Использование SudachiDict под GPL v3 не обязывает раскрывать исходный код собственного приложения при сервисной модели распространения (Software as a Service), поскольку распределение (distribution) в смысле GPL предполагает передачу бинарного кода пользователю, тогда как при SaaS пользователь взаимодействует с сервисом через API, а не получает исполняемый файл. Данная интерпретация соответствует позиции Фонда свободного программного обеспечения (FSF) [35] и подтверждается судебной практикой. Тем не менее в случае распространения исходного кода или бинарного приложения рекомендуется перейти на словарь UniDic-MeCab (BSD-3-Clause) или NTT UniDic (CC BY-SA).

3.3.3 Обоснование выбора лицензии MIT для разрабатываемого продукта

Для разрабатываемой платформы выбрана лицензия MIT. Обоснование выбора определяется тремя факторами. Во-первых, MIT — наиболее распространённая лицензия в экосистеме JavaScript и Python-инструментов,

что упрощает интеграцию с внешними библиотеками и понятно потенциальным контрибьюторам. Во-вторых, MIT не накладывает ограничений на коммерческое использование, что оставляет открытой возможность монетизации (SaaS, белый ярлык) без обязательства открывать доработки. В-третьих, MIT требует только сохранения текста уведомления об авторских правах в дистрибутиве, что выполнимо добавлением единственного файла LICENSE в корень репозитория.

3.3.4 Меры защиты авторских прав

Защита авторских прав на разработанное программное обеспечение обеспечивается следующими мерами. В каждом исходном файле в верхнем колонтитуле указывается стандартное уведомление об авторском праве в форме «Copyright (c) 2025 [Автор]. Licensed under the MIT License». Файл LICENSE в корне репозитория содержит полный текст лицензии MIT с указанием автора и года. Файл README.md содержит раздел «Лицензия» со ссылкой на LICENSE и перечнем основных компонентов с их лицензиями.

Дополнительным инструментом является государственная регистрация программы для ЭВМ в Федеральной службе по интеллектуальной собственности (Роспатент) в соответствии с Гражданским кодексом Российской Федерации (статья 1262). Регистрация не является обязательной, однако создаёт доказательную базу для разрешения споров об авторстве: свидетельство о государственной регистрации программы для ЭВМ закрепляет дату создания и личность автора. Процедура регистрации предполагает подачу заявки с описанием программы, распечаткой исходного кода (первые и последние 25 листов) и уплату государственной пошлины [36].

3.4 Аспекты информационной безопасности

3.4.1 Актуальные угрозы и реализованные меры защиты

Анализ угроз для разработанного приложения выполнен на основе классификации OWASP Top 10 (2021) [29] — наиболее распространённого

стандарта описания рисков безопасности веб-приложений. Из десяти категорий выбраны семь, наиболее релевантных для системы с публичным REST API, пользовательской аутентификацией и обработкой текстовых данных. Перечень угроз, оценка рисков и реализованные меры защиты приведены в таблице 3.8.

Таблица 3.8 – Угрозы OWASP Top 10 и реализованные меры защиты

Угроза (OWASP Top 10 2021)	ID	Уровень риска для системы	Реализованная мера защиты
Нарушение контроля доступа (Broken Access Control)	A01	Высокий	JWT-аутентификация с проверкой на каждом маршруте; защищённые API Routes в Next.js BFF; пользователь имеет доступ только к собственным данным (user_id фильтр в SQL-запросах)
Сбои криптографии (Cryptographic Failures)	A02	Средний	Пароли хранятся в виде bcrypt-хэша (cost factor 12); чувствительные параметры передаются исключительно через переменные окружения; HTTPS обязателен в production-конфигурации
Инъекции (Injection)	A03	Высокий	Все входные данные валидируются Pydantic-моделями до передачи в логику; ORM SQLAlchemy с параметризованными запросами исключает SQL-инъекции; длина строки запроса ограничена 1 000 символов
Небезопасная конфигурация	A05	Средний	CORS ограничен явным списком разрешённых источников (origins); в production-режиме отладочный режим FastAPI отключён; заголовки

Угроза (OWASP Top 10 2021)	ID	Уровень риска для системы	Реализованная мера защиты
(Security Misconfiguration)			безопасности (X-Content-Type-Options, X-Frame-Options) настроены через Next.js headers()
Уязвимые и устаревшие компоненты (Vulnerable and Outdated Components)	A06	Средний	Зависимости зафиксированы в requirements.txt и package-lock.json с явными версиями; GitHub Dependabot настроен для уведомления об уязвимостях CVE
Межсайтовый скриптинг (XSS)	A07	Средний	React экранирует пользовательские данные по умолчанию; dangerouslySetInnerHTML не используется; CJK-текст отображается через текстовые узлы DOM без HTML-интерпретации

3.4.2 Детальное описание реализованных мер

Валидация и санитизация входных данных реализована на уровне Pydantic-моделей FastAPI. Любой запрос к /analyze или /vocabulary, не удовлетворяющий схеме (неверный тип поля, отсутствие обязательного параметра, превышение допустимой длины), отклоняется с HTTP 422 до передачи в прикладную логику. Строки, передаваемые в токенизатор, предварительно санитизируются: удаляются управляющие символы Unicode (категории Cc, Cf) во избежание некорректной обработки SudachiPy или HanLP.

JWT-аутентификация с ограниченным сроком действия. Токен доступа имеет TTL 15 минут, refresh-токен — 7 суток, передаётся исключительно через httpOnly cookie, недоступный скриптам страницы. Алгоритм подписи — HS256 с секретом, передаваемым через переменную окружения JWT_SECRET; в production-среде рекомендуется переход на RS256 с парой RSA-ключей.

Настройка HTTPS и CORS. В production-конфигурации все HTTP-запросы перенаправляются на HTTPS через Nginx reverse proxy. Список разрешённых источников CORS передаётся через переменную окружения ALLOWED_ORIGINS и не содержит wildcard-значение в production. Заголовки безопасности настроены через Next.js headers() в next.config.js: Strict-Transport-Security (HSTS) со значением max-age=31536000, X-Content-Type-Options: nosniff, X-Frame-Options: DENY.

Ограничение частоты запросов (rate limiting). Для предотвращения злоупотреблений эндпоинтами /analyze и /explain реализован rate limiter на базе Redis [46]: не более 60 запросов в минуту с одного IP-адреса для анонимных пользователей и 120 запросов в минуту для аутентифицированных. При превышении лимита возвращается HTTP 429 с заголовком Retry-After.

Хранение паролей. Пароли пользователей хешируются алгоритмом bcrypt с cost factor 12, что обеспечивает время хеширования около 300–500 мс и делает атаки перебором по словарю практически неосуществимыми. В базе данных хранится исключительно хеш; исходный пароль нигде не записывается.

3.4.3 Вывод об уровне безопасности приложения

Реализованный набор мер защиты обеспечивает базовый уровень безопасности, соответствующий рекомендациям OWASP для веб-приложений [48], предназначенных для публичного доступа. Наиболее критичные категории — нарушение контроля доступа (A01) и инъекции (A03) — закрыты

на архитектурном уровне через ORM с параметризованными запросами и JWT-проверку на каждом маршруте. Отсутствующим компонентом остаётся аудит-журнал (logging) привилегированных действий (удаление записей, смена пароля), который рекомендуется добавить при подготовке системы к производственному развёртыванию.

3.5 Выводы по главе 3

По результатам третьей главы сформулированы следующие выводы. Функциональное тестирование подтвердило работоспособность системы: 12 функциональных сценариев из 12 и 7 граничных сценариев из 7 пройдены успешно. Нагрузочное тестирование показало, что требование НФТ-1 (время ответа не более 2 с) выполняется при числе одновременных запросов до 20, что соответствует ожидаемой нагрузке для учебного инструмента. Сравнительная таблица «требования — результат» зафиксировала выполнение 9 из 10 функциональных требований; единственное частично реализованное требование (семантический поиск через pgvector в ФТ-5) не влияет на основную функциональность и намечено к реализации в следующей версии. Покрываемость кода автоматизированными тестами составила 67%, что превышает пороговый показатель НФТ-6. Разработанный продукт публикуется под лицензией MIT, совместимой со всеми компонентами стека; единственная GPL-компонента (словарь SudachiDict) используется в сервисной модели, не требующей раскрытия исходного кода. Реализованные меры безопасности — JWT-аутентификация, bcrypt-хеширование паролей, ORM-параметризация запросов, CORS, rate limiting и заголовки безопасности — обеспечивают защиту от наиболее распространённых угроз категорий A01, A02, A03 и A07 по классификации OWASP. Направлениями дальнейшего развития системы являются: добавление корейского языка (хангыль) и интеграции с TOPIC-уровнями; реализация встроенной системы интервальных повторений (SRS) по алгоритму SM-2 непосредственно в личном словаре; разработка OCR-модуля для распознавания иероглифов с фотографий с использованием

предобученной модели компьютерного зрения; расширение AI-объяснений до полного грамматического разбора предложения с явным указанием синтаксических ролей каждого токена.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы были решены все поставленные задачи:

1. проведён анализ предметной области изучения иероглифической письменности: выявлены лингвистические особенности японского и китайского языков, охарактеризованы сложности освоения кандзи и ханьцзы, описана модель AS-IS учебного процесса учащегося без средств автоматизации;
2. выполнен сравнительный анализ пяти программных аналогов (Jisho, WaniKani, Duolingo, Anki, Pleco), в результате которого установлена незаполненная ниша: ни один из рассмотренных инструментов не совмещает морфологический анализ, контекстный поиск, AI-объяснения грамматики и открытый русскоязычный доступ;
3. сформулированы десять функциональных и шесть нефункциональных требований; определены языковые профили (японский и китайский режимы) как самостоятельные конфигурации интерфейса и данных;
4. спроектирована трёхзвенная клиент-серверная архитектура системы (Next.js 14 на клиенте, Python FastAPI на сервере, PostgreSQL 15 с pg_trgm и pgvector в качестве СУБД); разработаны ER-модель из четырёх таблиц и алгоритм эвристической классификации запросов, разграничивающий четыре класса входных данных без участия пользователя;
5. реализована серверная часть приложения: NLP-пайплайны на основе SudachiPy (японский) и HanLP (китайский), REST API из одиннадцати эндпоинтов с Pydantic-валидацией и двухуровневым кэшированием, JWT-аутентификация с refresh-токеном;
6. реализована клиентская часть: компоненты SentenceAnalyzer, WordInspector, KanjiInspector и DifficultyMeter с Effector-стейт-менеджментом, PWA-конфигурация с service worker;

7. реализован NLP-модуль с алгоритмом морфологического анализа, алгоритмом оценки сложности текста на основе медианы уровней токенов и интеграцией внешних API (Jisho, CC-CEDICT, OpenRouter);
8. проведено функциональное и нагрузочное тестирование: 12 из 12 функциональных и 7 из 7 граничных сценариев пройдены успешно; требование по времени ответа (не более 2 с) выполняется при числе одновременных запросов до 20.

Поставленная цель — разработка веб-платформы интеллектуального анализа и изучения иероглифических языков — достигнута. Разработанная система обеспечивает автоматическую обработку пользовательских запросов на японском и китайском языках, морфологический разбор с чтениями и переводами, инспектор иероглифов, оценку сложности текста по шкалам JLPT и HSK, AI-объяснения грамматических конструкций и ведение личного словаря — всё в рамках единого русскоязычного интерфейса без необходимости явного выбора языка или режима поиска.

Направлениями дальнейшего развития системы являются: разработка OCR-модуля для распознавания иероглифов с фотографий на основе предобученной модели компьютерного зрения; добавление поддержки корейского языка с интеграцией хангыля и уровней TOPIK; реализация встроенной системы интервальных повторений (SRS) по алгоритму SM-2 непосредственно в разделе личного словаря; публикация нативных мобильных приложений для iOS и Android на основе React Native с сохранением общей бизнес-логики серверной части.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. The Japan Foundation. Survey Report on Japanese-Language Education Abroad 2021. — Tokyo: The Japan Foundation, 2022. — URL: <https://www.jpf.go.jp/j/project/japanese/survey/result/survey21.html> (дата обращения: 12.05.2026).
2. Hanban / Confucius Institute Headquarters. HSK Examination Annual Report 2023. — Beijing: Hanban, 2024. — URL: <https://www.chinesetest.cn> (дата обращения: 12.05.2026).
3. HolonIQ. Global EdTech Market to reach \$404B by 2025. HolonIQ Intelligence Brief. — 2021. — URL: <https://www.holoniq.com/notes/global-edtech-market-to-reach-404b-by-2025> (дата обращения: 12.05.2026).
4. Coulmas F. Writing Systems of the World. — Oxford: Blackwell, 1989. — 302 p.
5. Shibatani M. The Languages of Japan. — Cambridge: Cambridge University Press, 1990. — 411 p.
6. Norman J. Chinese. — Cambridge: Cambridge University Press, 1988. — 292 p.
7. DeFrancis J. The Chinese Language: Fact and Fantasy. — Honolulu: University of Hawaii Press, 1984. — 330 p.
8. The Japan Foundation. Japanese-Language Proficiency Test: Official Practice Workbook. — Tokyo: Bonjinsha, 2022. — URL: <https://www.jlpt.jp> (дата обращения: 12.05.2026).

9. Hanban. HSK Standard Course. Level 6. — Beijing: Beijing Language and Culture University Press, 2015. — URL: <https://www.chinesetest.cn> (дата обращения: 12.05.2026).
10. Manning C. D., Schütze H. Foundations of Statistical Natural Language Processing. — Cambridge: MIT Press, 1999. — 680 p.
11. Godwin-Jones R. Emerging Technologies: Digital Literacies and Language Learning // Language Learning & Technology. — 2015. — Vol. 19, No. 3. — P. 8–19.
12. Ebbinghaus H. Memory: A Contribution to Experimental Psychology / transl. H. A. Ruger, C. E. Bussenius. — New York: Teachers College, Columbia University, 1913. — 123 p.
13. Wozniak P. A. Two components of long-term memory // Acta Neurobiologiae Experimentalis. — 1995. — Vol. 55. — P. 301–305.
14. Cepeda N. J., Pashler H., Vul E. et al. Distributed practice in verbal recall tasks: A review and quantitative synthesis // Psychological Bulletin. — 2006. — Vol. 132, No. 3. — P. 354–380.
15. Clegg D., Barker R. Case Method Fast-Track: A RAD Approach. — Wokingham: Addison-Wesley, 1994. — 240 p.
16. npm. Package download statistics. — URL: <https://www.npmjs.com/package/react> (дата обращения: 10.05.2026).

17. Ramírez S. FastAPI documentation. Performance. — 2024. — URL: <https://fastapi.tiangolo.com/benchmarks/> (дата обращения: 10.05.2026).

18. Bird S., Klein E., Loper E. Natural Language Processing with Python. — Sebastopol: O'Reilly Media, 2009. — 504 p.

19. Ramírez S. FastAPI: Modern, fast (high-performance) web framework for building APIs with Python 3.8+. — 2024. — URL: <https://fastapi.tiangolo.com> (дата обращения: 10.05.2026).

20. WorksApplications. SudachiPy: Python wrapper for Sudachi, a Japanese morphological analyzer. — URL: <https://github.com/WorksApplications/SudachiPy> (дата обращения: 10.05.2026).

21. Luo R., Xu J., Zhang Y. et al. PKUSEG: A toolkit for multi-domain Chinese word segmentation // arXiv:1906.11455. — 2019. — URL: <https://arxiv.org/abs/1906.11455> (дата обращения: 10.05.2026).

22. PostgreSQL Global Development Group. PostgreSQL 15 Documentation: Chapter 12. Full Text Search. — 2023. — URL: <https://www.postgresql.org/docs/15/textsearch.html> (дата обращения: 10.05.2026).

23. Sweller J. Cognitive load during problem solving: Effects on learning // Cognitive Science. — 1988. — Vol. 12, No. 2. — P. 257–285.

24. Mayer R. E. Multimedia Learning. 2nd ed. — Cambridge: Cambridge University Press, 2009. — 320 p.

25. FastAPI Documentation. Concurrency and async/await. — URL: <https://fastapi.tiangolo.com/async/> (дата обращения: 10.05.2026).
26. WorksApplications. SudachiPy: Python wrapper for Sudachi. — URL: <https://github.com/WorksApplications/SudachiPy> (дата обращения: 10.05.2026).
27. HanLP. hanlp: Han Language Processing. — URL: <https://hanlp.hankcs.com> (дата обращения: 10.05.2026).
28. Fielding R. T. Architectural Styles and the Design of Network-based Software Architectures: PhD dissertation. — Irvine: University of California, 2000.
29. OWASP Foundation. OWASP Top Ten 2021. — URL: <https://owasp.org/Top10/> (дата обращения: 10.05.2026).
30. Next.js Documentation. App Router. — URL: <https://nextjs.org/docs/app> (дата обращения: 10.05.2026).
31. Effector Documentation. Stores and Events. — URL: <https://effector.dev/docs> (дата обращения: 10.05.2026).
32. Myers G. J., Sandler C., Badgett T. The Art of Software Testing. 3rd ed. — Hoboken: Wiley, 2011. — 240 p.
33. Google Developers. Lighthouse Documentation. — URL: <https://developer.chrome.com/docs/lighthouse> (дата обращения: 10.05.2026).

34. pytest Documentation. Getting Started. — URL: <https://docs.pytest.org> (дата обращения: 10.05.2026).
35. Free Software Foundation. Does the GPL require that source code of modified versions be posted to the public? — URL: <https://www.gnu.org/licenses/gpl-faq.html> (дата обращения: 10.05.2026).
36. Гражданский кодекс Российской Федерации. Часть четвёртая. Статья 1262. Государственная регистрация программ для ЭВМ и баз данных. — М.: Консультант Плюс, 2024.
37. Stuttard D., Pinto M. The Web Application Hacker's Handbook. 2nd ed. — Indianapolis: Wiley, 2011. — 912 p.
38. Sutton I. Process Risk and Reliability Management. 2nd ed. — Amsterdam: Elsevier, 2015. — 814 p.
39. Knuth D. E. The Art of Computer Programming. Vol. 3: Sorting and Searching. 2nd ed. — Reading: Addison-Wesley, 1998. — 780 p.
40. Hochreiter S., Schmidhuber J. Long short-term memory // Neural Computation. — 1997. — Vol. 9, No. 8. — P. 1735–1780.
41. Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding // arXiv:1810.04805. — 2018.
42. Vaswani A., Shazeer N., Parmar N. et al. Attention Is All You Need // Advances in Neural Information Processing Systems 30. — 2017. — P. 5998–6008.

43. Python Software Foundation. Python 3.11 Documentation. — URL: <https://docs.python.org/3.11/> (дата обращения: 10.05.2026).
44. TypeScript Documentation. TypeScript 5.4 Handbook. — URL: <https://www.typescriptlang.org/docs/> (дата обращения: 10.05.2026).
45. Docker Documentation. Docker Compose Overview. — URL: <https://docs.docker.com/compose/> (дата обращения: 10.05.2026).
46. Redis Documentation. Redis 7.0 Reference. — URL: <https://redis.io/docs/> (дата обращения: 10.05.2026).
47. GitHub Actions Documentation. — URL: <https://docs.github.com/en/actions> (дата обращения: 10.05.2026).
48. Open Web Application Security Project. OWASP ASVS 4.0.3: Application Security Verification Standard. — 2021. — URL: <https://owasp.org/www-project-application-security-verification-standard/> (дата обращения: 10.05.2026).
49. Chou P.-C. Writing Chinese Characters. — Taipei: National Palace Museum, 2016. — 188 p.
50. Yokomori T. Learning to distinguish Japanese morphemes. — Tokyo: Waseda University, 1996. — 52 p.
51. Nielsen J. Usability Engineering. — Boston: Academic Press, 1993. — 362 p.

52. Newman S. Building Microservices: Designing Fine-Grained Systems. 2nd ed. — Sebastopol: O'Reilly, 2021. — 616 p.

53. Fowler M. Patterns of Enterprise Application Architecture. — Boston: Addison-Wesley, 2002. — 560 p.

54. Levenshtein V. I. Binary codes capable of correcting deletions, insertions, and reversals // Soviet Physics Doklady. — 1966. — Vol. 10, No. 8. — P. 707–710.

55. Tailwind CSS Documentation. — URL: <https://tailwindcss.com/docs> (дата обращения: 10.05.2026).

56. Alembic Documentation. Tutorial. — URL: <https://alembic.sqlalchemy.org/en/latest/tutorial.html> (дата обращения: 10.05.2026).